

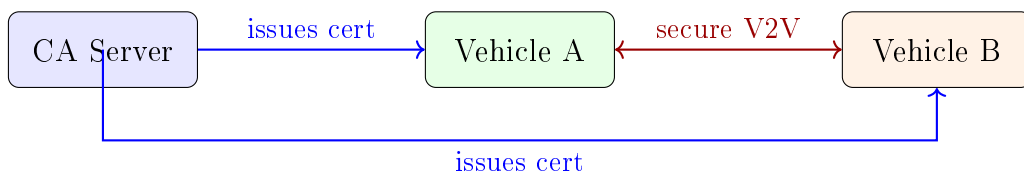
Vehicle-to-Vehicle (V2V)

Communication Security

A Complete Beginner's Guide to Theory and Practice

What You Will Learn

- V2V communication fundamentals and standards
- Cryptographic primitives with real-world analogies
- Public Key Infrastructure (PKI) from scratch
- Threat modelling and attack taxonomy
- Full Python implementation, step by step
- Attack demonstration and detection scripts
- Docker and two-laptop deployment guides



Prepared for Information Security Education
Complete Beginner to Practitioner Pathway

April 25, 2026

Abstract

This document is a comprehensive, end-to-end guide to understanding and implementing security for Vehicle-to-Vehicle (V2V) communication systems. It is written specifically for readers with **zero practical experience** in information security but who possess basic theoretical familiarity with computing concepts.

V2V communication allows vehicles on a road to exchange real-time safety messages — such as “I am braking hard” or “There is ice 300 metres ahead” — without passing through a central server. This makes V2V extraordinarily powerful for accident prevention but also extraordinarily dangerous if an attacker can forge, replay, or suppress those messages.

This guide proceeds in four major parts:

1. **Theory Foundation** — Every cryptographic primitive, security concept, and networking idea is explained from first principles using everyday analogies, then given its formal technical definition. No concept is assumed.
2. **Architecture and Design** — The full system blueprint is presented, with detailed diagram descriptions, Mermaid.js prompts, and TikZ code for every component.
3. **Implementation Guide** — Thirteen implementation steps walk through every module of the project with small, line-by-line annotated code segments and complete prompts for **Claude Code** to generate production-ready files.
4. **Reference and Appendices** — A glossary, standards bibliography, command cheat sheet, and troubleshooting guide serve as a permanent reference.

By the end of this document you will be able to explain V2V security to a technical audience, build a working simulation with certificate-based mutual authentication, AES-256-GCM encrypted messages, and replay/tamper detection — and demonstrate real attack scenarios being detected and blocked in real time.

Target audience: Undergraduate/postgraduate students, junior engineers, or hobbyists entering the field of vehicular network security.

Prerequisites: Basic Python familiarity, understanding of what a network packet is, and access to a Linux/macOS/WSL terminal.

Contents

Abstract	1
I Theory Foundation	9
1 What is Vehicle-to-Vehicle (V2V) Communication?	10
1.1 The Big Picture	10
1.2 How Cars Talk to Each Other — Technical Standards	11
1.2.1 DSRC — Dedicated Short-Range Communications	11
1.2.2 C-V2X — Cellular Vehicle-to-Everything	11
1.3 Use Cases: What Can V2V Actually Do?	12
2 Why Is Security Critical in V2V?	13
2.1 What Happens Without Security?	13
2.2 The CIA Triad Applied to V2V	13
2.2.1 Confidentiality in V2V	13
2.2.2 Integrity in V2V	14
2.2.3 Availability in V2V	14
3 Cryptographic Primitives	15
3.1 Symmetric Encryption — AES-256-GCM	15
3.2 Asymmetric Encryption — RSA and ECC	16
3.3 Digital Signatures — ECDSA	17
3.4 Key Exchange — ECDH	18
3.5 Hash Functions — SHA-256	18
3.6 MAC and HMAC — Message Authentication Codes	19
3.7 Nonces and Replay Prevention	20
4 Public Key Infrastructure and Certificate Authorities	21
4.1 What is a Certificate Authority?	21
4.2 X.509 Certificates — Field by Field	21

4.3	Certificate Chain of Trust	22
4.4	Pseudonymous Certificates in V2V	22
5	V2V Security Threat Model	24
5.1	Replay Attacks	24
5.2	Message Tampering / Man-in-the-Middle Attacks	25
5.3	Sybil Attacks	25
5.4	Denial of Service Attacks	26
5.5	Eavesdropping	26
6	Authentication Protocols in V2V	27
6.1	What is Mutual Authentication?	27
6.2	Challenge-Response Protocols	27
6.3	Session Keys and Forward Secrecy	28
6.4	The Three-Way Authentication Handshake	28
II	Project Architecture and Design	29
7	Project Overview and Goals	30
7.1	What We Are Building	30
7.2	Security Properties Achieved	30
8	Technical Stack	31
8.1	Language: Python 3.x	31
8.2	Cryptographic Libraries	31
8.2.1	cryptography (PyCA)	31
8.2.2	PyCryptodome	31
8.3	Network Layer	32
8.3.1	TCP Sockets	32
8.3.2	MQTT (Message Queuing Telemetry Transport)	32
8.4	Web Dashboard: Flask	33
9	Detailed Architecture Diagrams	34
9.1	D1 — Overall System Architecture	34
9.2	D2 — Authentication Handshake Sequence	35
9.3	D3 — Cryptographic Message Structure	36
9.4	D4 — Threat Model Diagram	36
9.5	D5 — PKI Certificate Chain	36
9.6	D6 — MQTT Topic Structure	37
9.7	D7 — Flask Dashboard Wireframe	37

9.8	D8 — Docker Network Topology	38
9.9	Project Folder Structure	38
III	Complete Project Implementation Guide	40
10	Step 1 — Environment Setup	41
10.1	Theory: Why Virtual Environments?	41
10.2	Setup Commands	41
10.3	requirements.txt	42
10.4	Two-Vehicle Configuration on One Machine	42
10.5	Two-Laptop Setup	43
11	Step 2 — Certificate Authority Implementation	44
11.1	Theory: What the CA Does	44
11.2	Small Code Segment: Key Generation	44
11.3	Full Claude Code Prompt for Step 2	45
11.4	Expected Output	46
12	Step 3 — Cryptographic Module	47
12.1	Theory: Why AES-GCM + ECDH + ECDSA Together?	47
12.2	Small Code Segment: AES-GCM Encrypt/Decrypt	47
12.3	Full Claude Code Prompt for Step 3	49
13	Step 4 — Authentication Protocol	51
13.1	Theory: The Handshake in Detail	51
13.2	Small Code Segment: HELLO Message	51
13.3	Full Claude Code Prompt for Step 4	52
14	Step 5 — V2V Message Protocol	55
14.1	Theory: Basic Safety Messages (BSM)	55
14.2	Small Code Segment: BSM Structure	56
14.3	Full Claude Code Prompt for Step 5	57
15	Step 6 — Main Vehicle Node	60
15.1	Theory: Socket Programming and Threading	60
15.2	Small Code Segment: Socket Server	60
15.3	Full Claude Code Prompt for Step 6	62
16	Step 7 — Flask Dashboard	64
16.1	Theory: Flask REST API	64
16.2	Full Claude Code Prompt for Step 7	64

17 Step 8 — Attack Demonstration Scripts	66
17.1 8a — Replay Attack	66
17.2 8b — Tamper Attack	67
17.3 8c — Man-in-the-Middle Attack	68
18 Step 9 — Testing Guide	71
18.1 Running the Full Demo	71
18.2 Expected Successful Authentication Output	72
18.3 Evaluator Demo Script	73
19 Step 10 — Security Evaluation Metrics	74
19.1 Measurement Methodology	75
19.2 Full Claude Code Prompt — Evaluation Script	76
20 Step 11 — MQTT Network Setup	78
20.1 Theory: MQTT vs TCP Sockets	78
20.2 MQTT Setup	78
20.3 Full Claude Code Prompt — MQTT Vehicle Node Variant	79
21 Step 12 — Docker Setup	80
21.1 Theory: What Docker Provides	80
21.2 Dockerfile	80
21.3 docker-compose.yml	81
21.4 Docker Commands	82
22 Step 13 — Two-Laptop Setup Guide	84
22.1 Network Configuration	84
22.2 Troubleshooting Common Issues	86
IV Reference and Appendices	87
A Glossary	88
B Standards References	91
C Further Reading	92
D Quick-Reference Command Cheat Sheet	93
E Troubleshooting Guide	96

List of Figures

4.1	Certificate Chain of Trust in V2V PKI	22
5.1	Replay Attack — Captured Message Retransmission	24
6.1	V2V Four-Step Authentication Handshake	28
9.1	D1: Overall V2V System Architecture	34
9.2	D3: Cryptographic Layering of a V2V BSM (Sign-then-Encrypt)	36
9.3	D4: Threat Model — Attack Vectors and Defences	36
9.4	D7: Flask Dashboard Wireframe	37
9.5	D8: Docker Container Network Topology	38

List of Tables

1.1	DSRC vs C-V2X Comparison	12
2.1	CIA Triad Applied to V2V	14
3.1	RSA vs ECC Key Size Equivalence	17
4.1	X.509 Certificate Fields Explained	22
7.1	Security Properties and How We Achieve Them	30
9.1	D6: MQTT Topic Structure	37
12.1	Cryptographic Algorithm Selection Rationale	47
19.1	Security Evaluation Metrics	75
20.1	MQTT vs TCP Sockets for V2V Simulation	78
22.1	Common Errors and Solutions	86
B.1	Key V2V Security Standards	91

Part I

Theory Foundation

Chapter 1

What is Vehicle-to-Vehicle (V2V) Communication?

1.1 The Big Picture

Imagine you are driving on a motorway at 100 km/h. A lorry three cars ahead of you suddenly brakes. You cannot see it because of the cars in between. Normally you would only learn about the braking when the car directly in front of you brakes — by which time you might have less than one second to react.

Now imagine every vehicle on that road is wearing a tiny invisible radio badge that continuously broadcasts: “I am Vehicle XYZ. I am at this GPS location. I am travelling at 100 km/h. My brakes are *not* applied.” And then, within milliseconds of the lorry braking, every vehicle within 300 metres receives the message: “Emergency braking event detected!” Your car can begin braking automatically, or at least give you a loud warning, *before* you can even see the problem.

This is V2V communication in its simplest form.

Real-Life Analogy

The Walkie-Talkie Highway: Think of V2V as every car on the road being equipped with a walkie-talkie tuned to the same channel. Cars are constantly whispering safety updates to their neighbours. The crucial difference from real walkie-talkies is that V2V is *standardised* (all cars speak the same “language”), *extremely fast* (messages travel in under 100 milliseconds), and *operates automatically* without any human pressing a button.

1.2 How Cars Talk to Each Other — Technical Standards

Key Concept

Formal Definition: Vehicle-to-Vehicle (V2V) communication is a wireless ad-hoc network technology that enables direct, peer-to-peer exchange of safety-critical data between nearby vehicles without relying on cellular infrastructure or a central server. It is a subset of the broader category called *Vehicular Ad Hoc Networks (VANETs)*.

Two major competing standards govern how vehicles communicate:

1.2.1 DSRC — Dedicated Short-Range Communications

DSRC is based on the **IEEE 802.11p** standard (a variation of ordinary Wi-Fi adapted for high-speed vehicular environments). Key facts:

- Operates in the **5.9 GHz** frequency band
- Communication range: up to **1000 metres**
- Latency: under **100 milliseconds** (crucial for safety)
- The underlying security standard is **IEEE 1609.2**
- Supported by: early US mandates, many V2X test programmes

Real-Life Analogy

DSRC is like the **CB (Citizens Band) radio** that truck drivers have used for decades — it is a dedicated, always-on radio channel set aside just for vehicles, operating independently of mobile phone networks. If the cell tower goes down, DSRC still works.

1.2.2 C-V2X — Cellular Vehicle-to-Everything

C-V2X uses **4G LTE / 5G NR** technology. It has two modes:

- **Mode 3 (Network mode):** messages route through cellular base stations (like your phone calls)
- **Mode 4 (Sidelink / Direct mode):** vehicles communicate directly without a base station, similar to DSRC

Table 1.1: DSRC vs C-V2X Comparison

Feature	DSRC (802.11p)	C-V2X (LTE/5G)
Radio technology	Wi-Fi derivative	Cellular
Range (direct)	Up to 1000 m	Up to 500–1000 m
Latency	<10 ms	10–100 ms
Infrastructure needed	No (ad hoc)	No for Mode 4
Security standard	IEEE 1609.2	3GPP TS 33.185
Deployment maturity	High (older)	Growing (newer)

1.3 Use Cases: What Can V2V Actually Do?

1. **Forward Collision Warning (FCW):** A car in front brakes suddenly. Before you can see the brake lights, your car knows and warns you.
2. **Intersection Movement Assist (IMA):** At a blind intersection, cars announce their approach so drivers/autonomous systems can avoid collisions.
3. **Emergency Electronic Brake Light (EEBL):** Hard braking 10 cars ahead triggers a warning in all following vehicles simultaneously.
4. **Do Not Pass Warning (DNPW):** A vehicle warns you not to overtake because it detects oncoming traffic you cannot see.
5. **Emergency Vehicle Preemption:** An ambulance broadcasts its presence; other vehicles know to pull over.
6. **Traffic Flow Coordination:** Dense platooning of trucks with centimetre-level gap accuracy, saving fuel.

Key Concept

SAE J2735 — The Message Standard: All V2V messages follow the *Basic Safety Message (BSM)* format defined in SAE J2735. A BSM is a data packet broadcast 10 times per second containing: vehicle ID (pseudonymous), GPS position, speed, heading, acceleration, and brake status. Think of it as a vehicle’s continuously updated “Twitter post” to its neighbours.

Chapter 2

Why Is Security Critical in V2V?

2.1 What Happens Without Security?

Let us think through a simple attack scenario with no cryptography at all:

Attack Scenario — The Phantom Braking Car:

Without authentication, an attacker parks by the roadside with a laptop and a radio transmitter. They broadcast a fake BSM claiming to be a car 50 metres ahead, travelling at 100 km/h, with emergency braking engaged. Every vehicle within 300 metres receives this fake message. Dozens of cars brake hard. A pile-up ensues. The attacker caused a multi-car accident from a car park.

This is not a theoretical scenario. Researchers at the University of Michigan demonstrated similar attacks in controlled studies as early as 2011. The stakes are not data theft — they are **human lives**.

2.2 The CIA Triad Applied to V2V

Key Concept

The **CIA Triad** is the foundational framework of information security. It has three components: **C**onfidentiality, **I**ntegrity, and **A**vailability. Every security control we build maps to one or more of these three properties.

2.2.1 Confidentiality in V2V

What it means: Only authorised parties can read the message content.

V2V challenge: BSMs are broadcast to everyone in range — by design they are

public. However, we do not want to leak the driver’s identity or precise location over time (privacy concern). Also, some V2V messages (e.g., fleet management commands) must remain private.

How we achieve it: *Pseudonymous certificates* (vehicles use rotating fake IDs) and end-to-end encryption for sensitive messages.

2.2.2 Integrity in V2V

What it means: Messages cannot be altered without detection.

V2V challenge: An attacker who intercepts a “I am travelling at 80 km/h” message and changes it to “I am travelling at 20 km/h” could cause unnecessary braking.

How we achieve it: *Digital signatures* (ECDSA) and *message authentication codes* (HMAC).

2.2.3 Availability in V2V

What it means: The system keeps working even under attack.

V2V challenge: An attacker flooding the channel with fake messages can drown out legitimate ones (Denial of Service).

How we achieve it: Rate limiting, source authentication, and channel management protocols.

Table 2.1: CIA Triad Applied to V2V

CIA Property	V2V Threat	Cryptographic Defence
Confidentiality	Location tracking, identity exposure	Pseudonymous certs, encryption
Integrity	Message tampering, fake BSMs	ECDSA signatures, HMAC
Availability	DoS flooding, jamming	Rate limiting, misbehaviour detection

Chapter 3

Cryptographic Primitives

Real-Life Analogy

Cryptography as a Toolbox: If security is a fortress, cryptography is your toolbox of locks, seals, keys, and safes. Each tool solves a specific problem. No single tool solves all problems. A good security system uses the right tool for the right job.

3.1 Symmetric Encryption — AES-256-GCM

Real-Life Analogy

The Locked Box Analogy: Alice puts a message in a box and locks it with a padlock. She gives Bob the *same key* she used to lock it. Bob unlocks the box and reads the message. The same key locks AND unlocks. This is symmetric encryption — one shared key for both operations.

Key Concept

AES-256-GCM stands for:

- **AES** — Advanced Encryption Standard: the gold-standard block cipher, approved by NIST, used everywhere from HTTPS to Wi-Fi to hard drive encryption.
- **256** — Key length in bits. 256-bit keys make brute-force attacks computationally impossible (2^{256} combinations = more than atoms in the observable universe).
- **GCM** — Galois/Counter Mode: an encryption mode that provides both *confidentiality* (encrypts the data) AND *integrity* (produces an authentic-

tion tag that detects tampering). This is called Authenticated Encryption with Associated Data (AEAD).

Why AES-GCM for V2V?

- Hardware-accelerated on modern CPUs (AES-NI instructions): extremely fast
- Provides encryption AND integrity in one operation
- The 96-bit nonce prevents ciphertext repetition
- Industry standard (used in TLS 1.3, IEEE 1609.2)

Security Warning

Never reuse a nonce! In AES-GCM, reusing the same nonce with the same key completely destroys security — an attacker can recover the key. Always generate a fresh random nonce for every message.

3.2 Asymmetric Encryption — RSA and ECC

Real-Life Analogy

The Padlock and Key Analogy: Alice gives Bob an *open padlock* (her public key). Anyone can put a message in a box, snap the padlock shut, and send it to Alice. But only Alice has the key to open it (her private key). The padlock is the public key — share it freely. The key is the private key — guard it with your life.

Key Concept

Asymmetric Encryption uses a **key pair**:

- **Public key:** Share with everyone. Used to encrypt data TO you, or to verify signatures FROM you.
- **Private key:** Keep secret forever. Used to decrypt data sent TO you, or to create signatures.

The mathematical relationship: anything encrypted with the public key can *only* be decrypted with the matching private key, and vice versa. This works because of hard mathematical problems (factoring large primes for RSA; discrete logarithms on elliptic curves for ECC).

RSA vs ECC for V2V:

RSA with 2048-bit keys offers strong security but is slow and uses large keys — a problem when vehicles must process dozens of BSMs per second. ECC (Elliptic Curve Cryptography) achieves equivalent security with **much smaller keys**:

Table 3.1: RSA vs ECC Key Size Equivalence

RSA Key Size	ECC Key Size	Security Level
1024 bits	160 bits	80-bit
2048 bits	224 bits	112-bit
3072 bits	256 bits	128-bit

V2V uses ECC (specifically P-256 curve) because small keys mean small certificates, which fit in tiny BSM packets and verify quickly.

3.3 Digital Signatures — ECDSA

Real-Life Analogy

The Wax Seal Analogy: In medieval times, a king would press his unique signet ring into wax on an envelope. Anyone who received the letter could verify it was authentic (the seal pattern is unique to the king) and unmodified (a broken seal means tampering). ECDSA is the cryptographic equivalent: a vehicle signs its BSM with its private key, and any receiver can verify the signature using the vehicle’s public key — without needing the private key.

Key Concept

ECDSA (Elliptic Curve Digital Signature Algorithm) produces a signature (r, s) computed from:

1. The message (or its hash)
2. The signer’s private key
3. A random nonce k

Verification uses only the signer’s *public* key. Key properties:

- **Authentication:** only the holder of the private key could have produced that signature
- **Non-repudiation:** the signer cannot later deny signing the message
- **Integrity:** any modification to the message invalidates the signature

Security Warning

ECDSA nonce reuse is catastrophic. If the same random nonce k is used for two different signatures with the same private key, the private key can be *algebraically recovered* by an attacker. Sony's PlayStation 3 was hacked exactly this way. Always use a cryptographically secure random number generator.

3.4 Key Exchange — ECDH

Real-Life Analogy

The Paint-Mixing Analogy (Diffie-Hellman): Alice and Bob both start with the same public colour (say, yellow). Alice picks a secret colour (red) and mixes it with yellow, getting orange. Bob picks a secret colour (blue) and mixes it with yellow, getting green. They exchange their mixed colours openly. Alice mixes Bob's green with her secret red; Bob mixes Alice's orange with his secret blue. Both end up with the same final colour (brown). An eavesdropper sees yellow, orange, and green — but cannot compute the final brown without knowing either secret colour.

Key Concept

ECDH (Elliptic Curve Diffie-Hellman) allows two parties to establish a *shared secret* over a public, insecure channel, without ever transmitting the secret itself. The shared secret is then used to derive a symmetric session key for fast AES-GCM encryption.

ECDHE (with the “E” = Ephemeral) generates fresh key pairs for each session, providing *Forward Secrecy*: if today's session key is compromised, past and future sessions remain secure.

3.5 Hash Functions — SHA-256

Real-Life Analogy

The Fingerprint Analogy: Every human has unique fingerprints. A fingerprint is a compact representation that uniquely identifies a person, but you cannot reconstruct the person from their fingerprint. SHA-256 is a fingerprint for data: any file or message, no matter how large, produces a unique 256-bit (32-byte) “fingerprint” called a *digest* or *hash*. Change even one bit in the input and the hash changes completely.

Key Concept

A **cryptographic hash function** maps arbitrary-length input to fixed-length output with these properties:

- **Deterministic:** same input always gives same output
- **One-way (preimage resistance):** cannot reverse the hash to get input
- **Collision resistant:** computationally infeasible to find two different inputs with the same hash
- **Avalanche effect:** one-bit change in input $\Rightarrow$ completely different output

SHA-256 produces a 256-bit digest and is the NIST standard (SHA-2 family).

3.6 MAC and HMAC — Message Authentication Codes

Real-Life Analogy

The Tamper-Evident Seal Analogy: Imagine you order a package and it arrives with a sticker that reads “SEALED BY SENDER — 3847AB”. The code on the sticker was computed using a secret only you and the sender know. If anyone tampers with the package in transit, they cannot reproduce the correct code without the secret, so the sticker will not match. HMAC works the same way: it produces a tag using a shared secret key that proves both the message’s integrity AND the sender’s identity.

Key Concept

HMAC (Hash-based Message Authentication Code) is computed as:

$$\text{HMAC}(K, M) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M))$$

where H is a hash function (SHA-256), K is the shared key, M is the message, and ipad/opad are constants. HMAC provides:

- **Integrity:** any modification to M produces a different tag
- **Authentication:** only parties with K can produce a valid tag

Unlike digital signatures, HMAC requires a *shared* secret — both parties must already have the same key.

3.7 Nonces and Replay Prevention

Real-Life Analogy

The Unique Ticket Stub Analogy: When you buy a concert ticket, it has a unique barcode. The bouncer scans it; once used, it is marked in the system. If you try to use it again, or if someone copied your ticket and tries to use it later, the system rejects it because that stub has already been redeemed. A *nonce* in cryptography is a “Number used ONCE” — a unique random value attached to each message that the receiver records and rejects if seen again.

Key Concept

A **nonce** is a random or pseudo-random number that may only be used once in a cryptographic communication. Combined with **timestamps**, nonces prevent *replay attacks*: an attacker recording a valid message and replaying it later will be rejected because:

1. The nonce has already been seen (stored in a *replay cache*)
2. The timestamp is too old (outside the acceptable replay window, e.g., 5 seconds)

Chapter 4

Public Key Infrastructure and Certificate Authorities

4.1 What is a Certificate Authority?

Real-Life Analogy

The Government Passport Office Analogy: How do you prove to a foreign customs officer that you are who you say you are? You show a passport — a document issued and *signed* by your government, which the customs officer trusts. The government is the **Certificate Authority (CA)**; your passport is the **digital certificate**; the customs officer is a **receiving vehicle** verifying your identity. The CA's signature on your certificate is the cryptographic equivalent of the government's stamp.

Key Concept

A **Certificate Authority (CA)** is a trusted third party that:

1. Verifies the identity of an entity (vehicle, server, person)
2. Issues a **digital certificate** binding that entity's identity to their public key
3. Signs the certificate with the CA's own private key so anyone can verify its authenticity using the CA's public key

4.2 X.509 Certificates — Field by Field

An X.509 certificate is a standardised data structure. Here is what each field means:

Table 4.1: X.509 Certificate Fields Explained

Field	What It Means	Example
Version	Certificate format version	3
Serial Number	Unique ID assigned by CA	0xA4B2. . .
Issuer	Who signed this certificate	CN=V2V-Root-CA
Subject	Who this certificate belongs to	CN=Vehicle-001
Validity (Not Before)	Certificate starts being valid	2024-01-01
Validity (Not After)	Certificate expires	2025-01-01
Subject Public Key	The entity’s public key	ECDSA P-256 key
Signature Algorithm	How the CA signed it	SHA256withECDSA
CA Signature	The CA’s digital signature	(256-bit value)
Extensions	Extra info (key usage, etc.)	keyUsage: digitalSign

4.3 Certificate Chain of Trust

Real-world PKI uses a **hierarchy** of certificates:

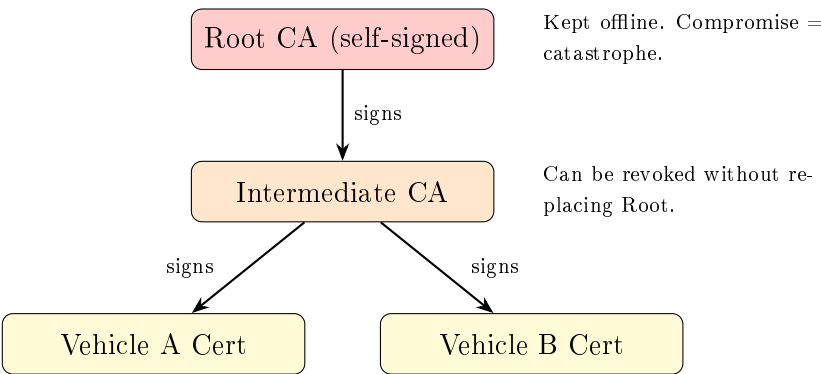


Figure 4.1: Certificate Chain of Trust in V2V PKI

4.4 Pseudonymous Certificates in V2V

Key Concept

A key privacy challenge in V2V: if every vehicle uses a *permanent* certificate with a fixed identifier, an observer can track that vehicle’s location indefinitely by correlating BSMs. To solve this, vehicles use **pseudonymous certificates**:

- Each vehicle is pre-loaded with many short-lived certificates (e.g., valid for 5 minutes each)
- Vehicles rotate certificates frequently
- The CA knows the link between pseudonyms and real identities (for mis-

behaviour reporting), but observers do not
This balances **privacy** (unlinkability for observers) with **accountability** (CA
can revoke misbehaving vehicles).

Chapter 5

V2V Security Threat Model

A **threat model** is a structured analysis of who might attack your system, what they want to achieve, what capabilities they have, and how you defend against each attack.

5.1 Replay Attacks

Real-Life Analogy

The Recorded Doorbell Analogy: A burglar records a neighbour's voice saying "Coming, just a moment!" Then later, when the security guard calls, the burglar plays the recording to pretend someone is home. Replay attacks work the same way: an attacker records a valid V2V message (e.g., "all clear, road is safe") and replays it later when conditions have changed.

How it works in V2V:

1. Attacker captures a valid, signed BSM from Vehicle A at time T_1
2. At time T_2 (later, when the message is no longer accurate), attacker retransmits the captured message unchanged
3. Without replay protection, receivers would accept it as genuine

Defence: Timestamps (reject messages older than 5 seconds) + Nonces (maintain a cache of recently seen nonces; reject duplicates).

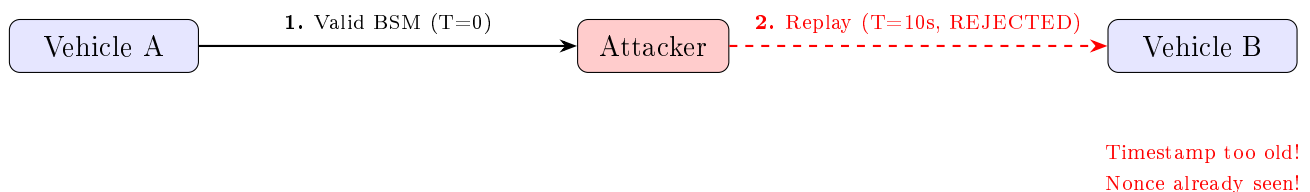


Figure 5.1: Replay Attack — Captured Message Retransmission

5.2 Message Tampering / Man-in-the-Middle Attacks

Real-Life Analogy

The Intercepted Letter Analogy: Alice sends Bob a letter through a postal service. A malicious postal worker opens the letter, changes the contents (e.g., changes the meeting time), reseals it, and forwards it. Without a signature or seal, Bob cannot know the letter was modified.

How it works in V2V:

1. Attacker intercepts a BSM between Vehicle A and Vehicle B
2. Attacker modifies the speed field from 100 km/h to 20 km/h
3. Attacker forwards the modified message

Defence: ECDSA digital signature covers the entire message. Any modification — even one bit — produces a different hash, invalidating the signature. Receiver detects the mismatch and discards the message.

5.3 Sybil Attacks

Real-Life Analogy

The Fake Crowd Analogy: Imagine an election where one person creates 100 fake identities and votes 100 times. A Sybil attack in V2V is when one attacker broadcasts from many fake vehicle identities simultaneously to create the illusion of a traffic jam (causing others to reroute) or to overwhelm the network.

Key Concept

In a **Sybil attack**, a single malicious node claims multiple fake identities. In V2V, an attacker with one radio transmitter could impersonate dozens of fake vehicles, causing:

- **Ghost traffic jams:** other vehicles reroute unnecessarily
- **Network flooding:** DoS via message overload
- **Map poisoning:** corrupting real-time traffic data

Defence: Each valid identity requires a valid certificate signed by the CA. Generating many fake identities requires obtaining many fake certificates — which the CA will not issue. The CA's verification process is the barrier.

5.4 Denial of Service Attacks

Real-Life Analogy

The Jammed Walkie-Talkie Channel Analogy: If someone holds down the transmit button on a walkie-talkie channel with static noise, nobody else can communicate. A radio jammer does the same to DSRC channels.

Types of V2V DoS:

- **Physical jamming:** radio frequency interference (hard to prevent)
- **Protocol flooding:** sending millions of BSMs to overwhelm receivers
- **Certificate flooding:** triggering expensive certificate verification for every fake message

Defence: Rate limiting per sender, lightweight verification steps, channel management, and misbehaviour detection systems.

5.5 Eavesdropping

Since V2V uses broadcast radio, *anyone* with the right antenna can receive BSMs. This enables:

- **Location tracking:** observing a vehicle's movement over time
- **Behaviour profiling:** deducing driver habits (commute routes, stops)

Defence: Pseudonymous certificates + certificate rotation + optional encryption for sensitive fields.

Chapter 6

Authentication Protocols in V2V

6.1 What is Mutual Authentication?

Real-Life Analogy

The Secret Handshake Analogy: Two spies meeting for the first time each need to prove they are who they claim to be. One spy shows a secret symbol; the other responds with the correct counter-symbol. Both authenticate each other simultaneously. This is **mutual** authentication — both parties verify the other's identity before trusting them.

Key Concept

Mutual Authentication means both parties in a communication session verify the other's identity before exchanging sensitive data. In V2V:

- Vehicle A proves to Vehicle B that it has a valid CA-issued certificate and the private key matching that certificate
- Vehicle B proves the same to Vehicle A
- Neither trusts the other until *both* proofs are verified

6.2 Challenge-Response Protocols

Key Concept

A **challenge-response protocol** works as follows:

1. **Challenge:** “If you really are Vehicle A with private key k_A , sign this random number r for me.”
2. **Response:** Vehicle A signs r with k_A and returns the signature.
3. **Verification:** The challenger verifies the signature using Vehicle A's public key (from its certificate). If valid, identity is proven.

This prevents impersonation because only the true holder of k_A can produce a valid signature on the challenge.

6.3 Session Keys and Forward Secrecy

Key Concept

A **session key** is a short-lived symmetric key used only for the duration of one communication session. After authentication, both vehicles derive the same session key using ECDH. This session key is then used for fast AES-GCM encryption.

Forward Secrecy (achieved with ECDHE) means: even if an attacker later obtains a vehicle's long-term private key, they cannot decrypt past sessions because each session used a fresh, ephemeral key pair that no longer exists.

6.4 The Three-Way Authentication Handshake

Our V2V authentication protocol follows this four-step flow:

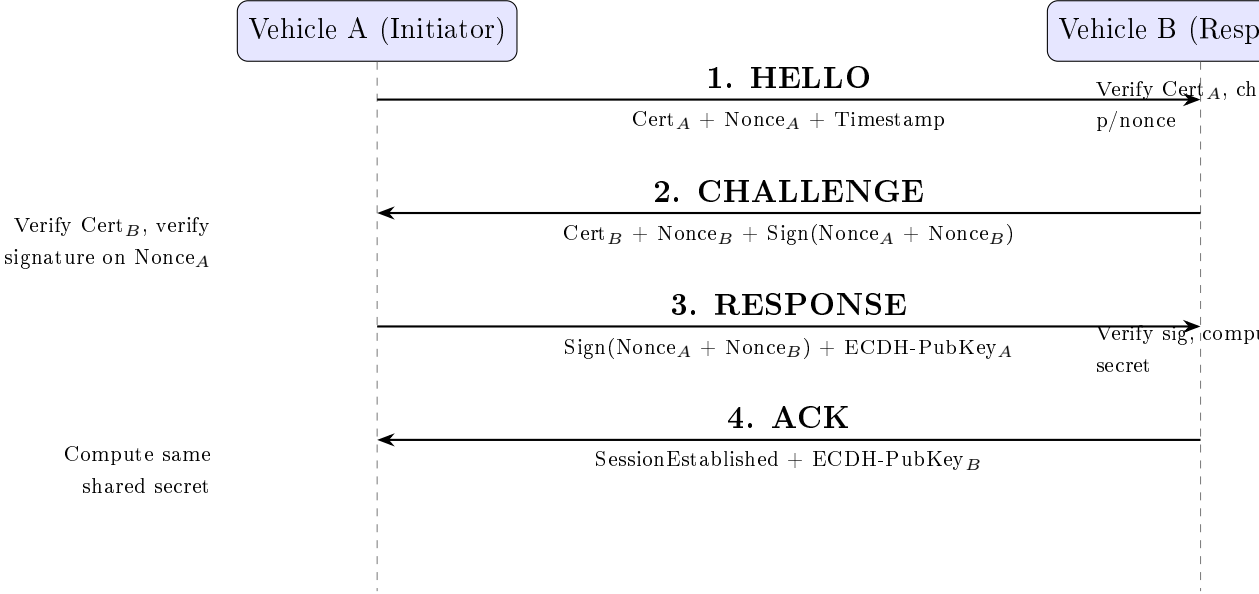


Figure 6.1: V2V Four-Step Authentication Handshake

Part II

Project Architecture and Design

Chapter 7

Project Overview and Goals

7.1 What We Are Building

We are building a **Python-based simulation of a secure V2V communication system** that demonstrates:

1. A **Certificate Authority** that issues digital certificates to vehicles
2. Two **Vehicle Nodes** that authenticate each other using those certificates
3. **Encrypted BSM exchange** using AES-256-GCM with ECDH session keys
4. A **Flask dashboard** showing authentication status, messages, and security properties in real time
5. **Attack demonstration scripts** showing replay, tampering, and MitM attacks being detected and blocked

7.2 Security Properties Achieved

Table 7.1: Security Properties and How We Achieve Them

Property	Description	Mechanism
Authentication	Both vehicles verify each other's identity	ECDSA + X.509 certs
Confidentiality	Messages encrypted in transit	AES-256-GCM
Integrity	Tampering is detected	ECDSA signature + HMAC
Non-repudiation	Sender cannot deny sending	ECDSA signature
Replay Prevention	Old messages rejected	Nonce cache + timestamps
Forward Secrecy	Past sessions safe if key leaked	ECDHE session keys

Chapter 8

Technical Stack

8.1 Language: Python 3.x

Key Concept

Why Python for security projects?

- **Readable:** Security logic is clearly visible; bugs are easier to spot
- **Rich ecosystem:** PyCA/cryptography library is maintained by security experts and audited by the community
- **Rapid prototyping:** Build working prototypes quickly to test design ideas
- **Education:** The code reads almost like pseudocode, perfect for learning

In production V2V systems, C++ or Rust would be used for performance. For our simulation, Python is ideal.

8.2 Cryptographic Libraries

8.2.1 cryptography (PyCA)

```
1 pip install cryptography
2 # This installs the PyCA cryptography library
3 # Provides: AES-GCM, ECDSA, ECDH, X.509, HMAC, SHA-256
4 # Maintained by: Python Cryptographic Authority
5 # Audited by: security professionals
6 # Used in: Let's Encrypt, Twisted, many production systems
```

Listing 8.1: Installing PyCA cryptography

8.2.2 PyCryptodome

```
1 pip install pycryptodome
2 # PyCryptodome provides additional cipher implementations
3 # and is a drop-in replacement for the older PyCrypto library.
4 # We use it for: DES (for demonstration of weak ciphers),
5 # additional modes, and RSA PKCS1 OAEP padding examples.
```

Listing 8.2: Installing PyCryptodome

8.3 Network Layer

8.3.1 TCP Sockets

Python’s built-in `socket` module provides TCP connectivity. For our simulation, TCP gives us:

- Reliable, ordered delivery (no packet loss in simulation)
- Simple server/client model
- Full duplex (both sides can send simultaneously)

8.3.2 MQTT (Message Queuing Telemetry Transport)

Real-Life Analogy

The Publish/Subscribe (Pub/Sub) Radio Station Analogy: In a traditional walkie-talkie conversation, you call one person. In MQTT, think of radio broadcasting: a radio station (publisher) broadcasts on a frequency (topic); anyone with a radio tuned to that frequency (subscriber) receives it. You do not need to know who is listening — you just broadcast. This is the **publish/subscribe model**.

Key Concept

MQTT is a lightweight messaging protocol designed for IoT devices. Key concepts:

- **Broker:** The central server that routes messages (like a radio relay tower)
- **Publisher:** Sends messages to a topic (Vehicle A publishes its BSMs)
- **Subscriber:** Receives messages on a topic (Vehicle B subscribes to Vehicle A’s BSM topic)
- **Topic:** A string path like `v2v/vehicle-a/bsm`
- **QoS levels:** 0 (fire and forget), 1 (at least once), 2 (exactly once)

We use **Eclipse Mosquitto** as our MQTT broker — free, open source, and widely used.

8.4 Web Dashboard: Flask

Key Concept

Flask is a lightweight Python web framework. It lets us create a simple HTTP server that:

1. Serves HTML pages to a browser
2. Provides REST API endpoints returning JSON data
3. Connects to our vehicle node to query current state

We use Flask because it requires minimal setup and zero external dependencies beyond the library itself.

Chapter 9

Detailed Architecture Diagrams

9.1 D1 — Overall System Architecture

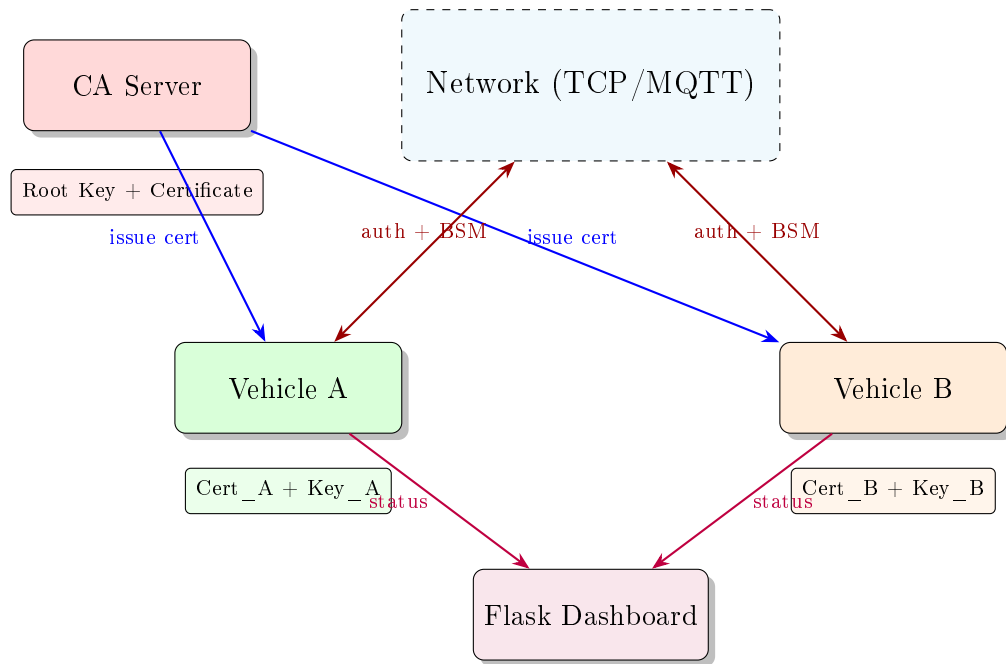


Figure 9.1: D1: Overall V2V System Architecture

Mermaid.js Prompt for D1

To regenerate D1 in Mermaid.js, paste the following into <https://mermaid.live>:

```
graph TB
  CA["CA Server\n(ca.py)"]
  VA["Vehicle A\n(vehicle_node.py)"]
  VB["Vehicle B\n(vehicle_node.py)"]
  NET["Network\n(TCP/MQTT)"]
  FLASK["Flask Dashboard\n(app.py)"]
```

```

CA -->|"issue cert"| VA
CA -->|"issue cert"| VB
VA <-->|"auth + BSM"| NET
VB <-->|"auth + BSM"| NET
VA -->|"status"| FLASK
VB -->|"status"| FLASK

```

9.2 D2 — Authentication Handshake Sequence

(See Figure 6.1 in Chapter 6 for the TikZ rendering.)

Mermaid.js Prompt for D2

```

sequenceDiagram
    participant A as Vehicle A
    participant B as Vehicle B

    A->>B: 1. HELLO {Cert_A, Nonce_A, Timestamp}
    Note over B: Verify Cert_A, check timestamp
    B->>A: 2. CHALLENGE {Cert_B, Nonce_B, Sign(Nonce_A|Nonce_B)}
    Note over A: Verify Cert_B, verify signature on Nonce_A
    A->>B: 3. RESPONSE {Sign(Nonce_A|Nonce_B), ECDH_PubKey_A}
    Note over B: Verify signature, compute shared secret
    B->>A: 4. ACK {SessionEstablished, ECDH_PubKey_B}
    Note over A: Compute shared secret -> Session Key

```

9.3 D3 — Cryptographic Message Structure

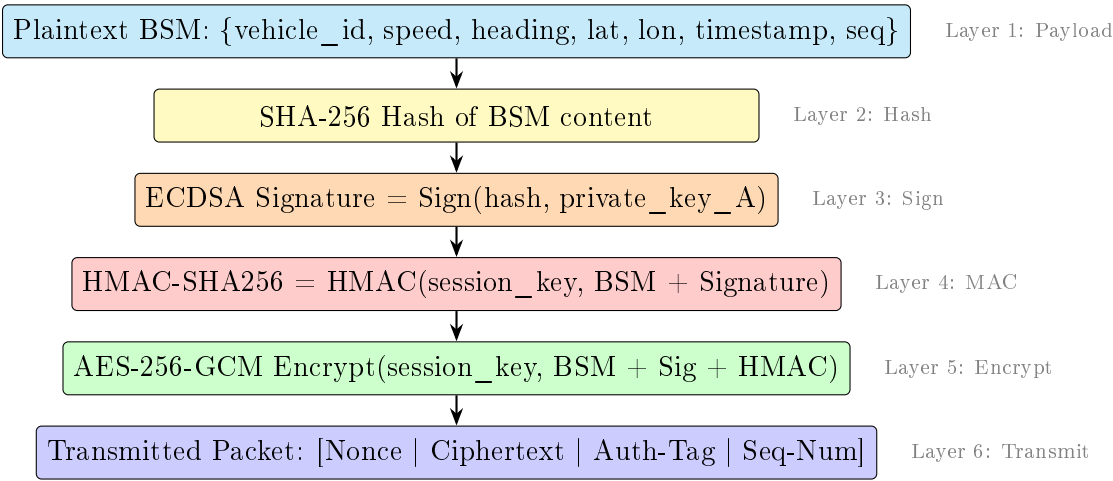


Figure 9.2: D3: Cryptographic Layering of a V2V BSM (Sign-then-Encrypt)

9.4 D4 — Threat Model Diagram

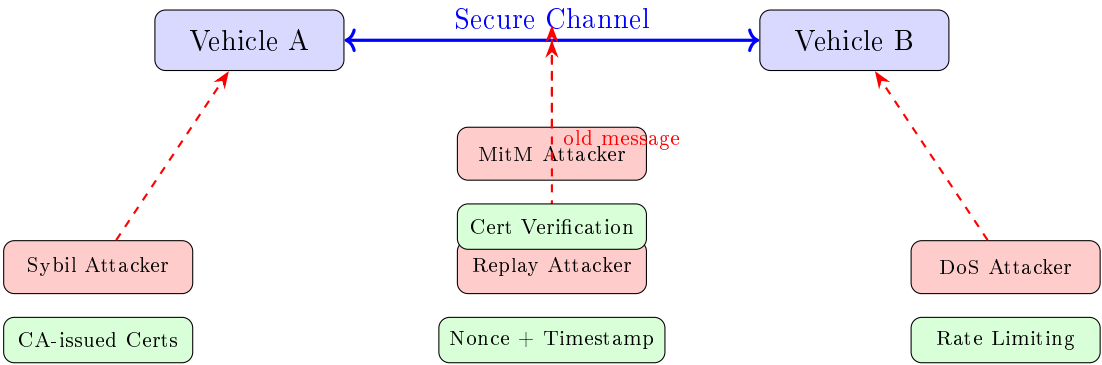


Figure 9.3: D4: Threat Model — Attack Vectors and Defences

9.5 D5 — PKI Certificate Chain

(See Figure 4.1 in Chapter 4.)

9.6 D6 — MQTT Topic Structure

Table 9.1: D6: MQTT Topic Structure

Topic	Purpose	Publisher	Subscriber
v2v/vehicle-a/bsm	BSMs from Vehicle A	Vehicle A	Vehicle B, Dashboard
v2v/vehicle-b/bsm	BSMs from Vehicle B	Vehicle B	Vehicle A, Dashboard
v2v/vehicle-a/auth	Auth messages	Vehicle A	Vehicle B
v2v/vehicle-b/auth	Auth messages	Vehicle B	Vehicle A
v2v/alerts	Attack alerts	Any node	Dashboard
v2v/+/bsm	All BSMs (wildcard)	—	Monitor tools

9.7 D7 — Flask Dashboard Wireframe

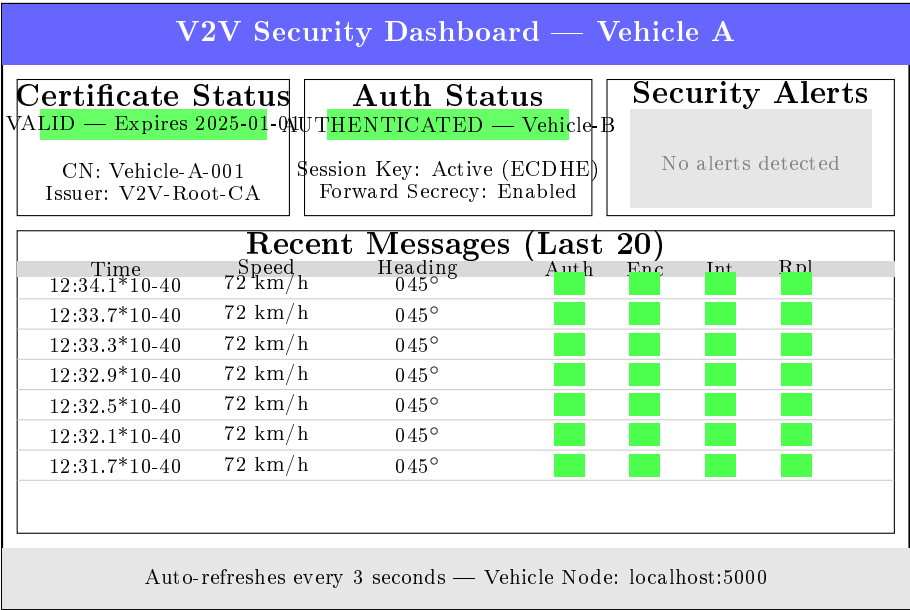


Figure 9.4: D7: Flask Dashboard Wireframe

9.8 D8 — Docker Network Topology

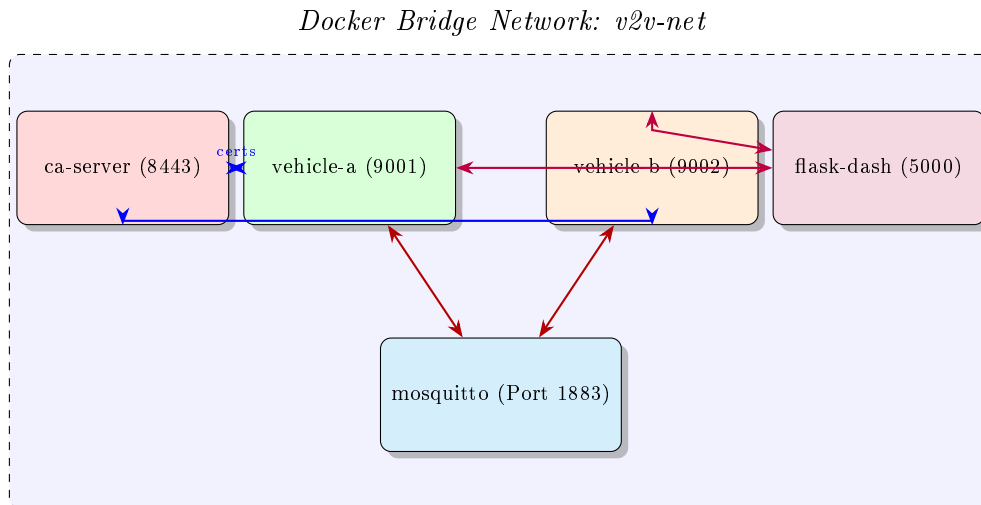


Figure 9.5: D8: Docker Container Network Topology

9.9 Project Folder Structure

```

v2v_security/
|-- ca/                                # Certificate Authority module
|   |-- ca.py                          # Root CA: generates key pair + self-
|   signed cert
|   |-- issue_cert.py                  # Issues signed certificates to vehicles
|   |-- verify_cert.py                 # Verifies certificate authenticity
|   |-- certs/                         # Generated certificates (PEM files)
|       |-- ca_cert.pem                # CA root certificate (share with all)
|       |-- ca_key.pem                 # CA private key (keep SECRET)
|       |-- vehicle_a_cert.pem
|       |-- vehicle_a_key.pem
|       |-- vehicle_b_cert.pem
|       |-- vehicle_b_key.pem
|
|-- crypto/                             # Cryptographic operations module
|   '-- crypto.py                      # AES-GCM, ECDH, ECDSA, HMAC, key
|   derivation
|
|-- protocol/                           # Communication protocol layer
|   |-- auth_protocol.py               # Handshake: HELLO/CHALLENGE/RESPONSE/
|   ACK
|   '-- v2v_protocol.py                # BSM: create, secure, send, receive,
|   verify
|
|-- node/                               # Vehicle node implementation

```



```

|   '-- vehicle_node.py      # Main vehicle class: TCP server + BSM
|   loop
|
|-- dashboard/              # Flask web dashboard
|   |-- app.py              # Flask app: REST API + dashboard server
|   '-- templates/
|       '-- dashboard.html  # HTML dashboard page
|
|-- attacks/                # Attack demonstration scripts
|   |-- replay_attack.py    # Captures BSM and replays it (shows
|       rejection)
|   |-- tamper_attack.py    # Modifies BSM in transit (shows sig
|       failure)
|   '-- mitm_attack.py     # Man-in-the-middle relay (shows cert
|       failure)
|
|-- tests/                  # Test suite
|   |-- test_crypto.py      # Unit tests for all crypto functions
|   |-- test_auth.py        # Integration tests for handshake
|   '-- test_v2v.py         # End-to-end message tests
|
|-- docker/                 # Docker configuration
|   |-- Dockerfile          # Image definition for vehicle nodes
|   '-- docker-compose.yml  # Multi-container orchestration
|
|-- requirements.txt         # All Python dependencies with versions
|-- README.md               # Project overview and quick-start guide
'-- config.py               # Shared configuration constants

```

Listing 9.1: Complete Project Directory Tree

Part III

Complete Project Implementation Guide

Chapter 10

Step 1 — Environment Setup

10.1 Theory: Why Virtual Environments?

Key Concept

A **virtual environment** is an isolated Python installation that has its own set of packages, separate from your system Python. This prevents version conflicts between projects and ensures your project's dependencies are fully reproducible. Think of it as giving your project its own clean workbench.

10.2 Setup Commands

```
# 1. Create project directory
mkdir v2v_security && cd v2v_security

# 2. Create virtual environment
python3 -m venv venv

# 3. Activate it
source venv/bin/activate          # Linux/macOS
# .\venv\Scripts\activate         # Windows PowerShell

# 4. Verify activation (should show venv path)
which python3

# 5. Install all dependencies
pip install cryptography pycryptodome paho-mqtt flask requests pytest
```

Listing 10.1: Environment setup — Linux/macOS

10.3 requirements.txt

```

1 # requirements.txt
2 # Install all with: pip install -r requirements.txt
3
4 cryptography==42.0.5      # PyCA: AES-GCM, ECDSA, ECDH, X.509, HMAC,
   SHA-256
5 pycryptodome==3.20.0     # Extra cipher implementations, legacy
   support
6 paho-mqtt==1.6.1         # MQTT client: publish/subscribe to
   Mosquitto broker
7 flask==3.0.3             # Lightweight web framework for the
   dashboard
8 requests==2.31.0         # HTTP client: dashboard polls vehicle
   node REST API
9 pytest==8.1.1            # Test runner for unit and integration
   tests
10 pytest-cov==4.1.0       # Test coverage reporting
11 python-dateutil==2.9.0   # Timestamp parsing and timezone handling
12 dataclasses-json==0.6.4  # JSON serialization for BSM dataclass

```

Listing 10.2: requirements.txt with explanations

10.4 Two-Vehicle Configuration on One Machine

```

# Vehicle A listens on port 9001, Vehicle B on port 9002
# Run in separate terminals:

# Terminal 1 --- Vehicle A
python node/vehicle_node.py \
  --vehicle-id vehicle-a \
  --port 9001 \
  --peer-host 127.0.0.1 \
  --peer-port 9002 \
  --ca-cert ca/certs/ca_cert.pem

# Terminal 2 --- Vehicle B
python node/vehicle_node.py \
  --vehicle-id vehicle-b \
  --port 9002 \
  --peer-host 127.0.0.1 \
  --peer-port 9001 \
  --ca-cert ca/certs/ca_cert.pem

```

Listing 10.3: Single-machine two-vehicle setup

10.5 Two-Laptop Setup

```
# Step 1: Find Laptop A's IP address
ip addr show          # Linux
ipconfig              # Windows

# Suppose Laptop A has IP: 192.168.1.100
# Suppose Laptop B has IP: 192.168.1.101

# Step 2: Open firewall port on both laptops (Linux)
sudo ufw allow 9001/tcp
sudo ufw allow 9002/tcp
sudo ufw allow 5000/tcp    # Flask dashboard

# Step 3: Copy certificates to Laptop B (via scp or USB)
scp ca/certs/ca_cert.pem user@192.168.1.101:~/v2v_security/ca/certs/
scp ca/certs/vehicle_b_cert.pem user@192.168.1.101:~/v2v_security/ca/
  certs/
scp ca/certs/vehicle_b_key.pem user@192.168.1.101:~/v2v_security/ca/
  certs/

# Step 4: Laptop A --- Run Vehicle A
python node/vehicle_node.py \
    --vehicle-id vehicle-a \
    --port 9001 \
    --peer-host 192.168.1.101 \    # Laptop B's IP
    --peer-port 9002 \
    --ca-cert ca/certs/ca_cert.pem

# Step 5: Laptop B --- Run Vehicle B
python node/vehicle_node.py \
    --vehicle-id vehicle-b \
    --port 9002 \
    --peer-host 192.168.1.100 \    # Laptop A's IP
    --peer-port 9001 \
    --ca-cert ca/certs/ca_cert.pem
```

Listing 10.4: Two-laptop network setup

Chapter 11

Step 2 — Certificate Authority Implementation

11.1 Theory: What the CA Does

The CA is the **trust anchor** of the entire system. It generates the root key pair and certificate, then issues signed certificates to vehicles. Any vehicle that trusts the CA automatically trusts every certificate the CA has signed.

Key Concept

CA Bootstrap Process:

1. CA generates its own ECDSA key pair (private + public)
2. CA creates a self-signed certificate (it signs its own certificate with its own private key — the “root of trust”)
3. When a vehicle requests a certificate, the CA verifies the vehicle’s identity and signs the vehicle’s certificate with the CA’s private key
4. Vehicles verify each other’s certificates by checking the CA’s signature using the CA’s public key (from the CA certificate)

11.2 Small Code Segment: Key Generation

```
1 from cryptography.hazmat.primitives.asymmetric.ec import (
2     generate_private_key,    # Function to create a new EC key pair
3     SECP256R1                # The P-256 curve (NIST standard, used
4                               in V2V)
5 )
6
7 from cryptography.hazmat.backends import default_backend
8
9 # Generate the CA's private key on the P-256 elliptic curve
```

```

8 # SECP256R1 = "SEC (Standards for Efficient Cryptography) P-256 R1"
9 # This is the same curve used in HTTPS and IEEE 1609.2
10 ca_private_key = generate_private_key(
11     curve=SECP256R1(),          # Specifies which curve to use
12     backend=default_backend() # Uses the platform's cryptographic
    backend
13 )
14
15 # Derive the public key from the private key
16 # Public keys can be freely shared; never share the private key
17 ca_public_key = ca_private_key.public_key()
18
19 # At this point we have a key PAIR:
20 # ca_private_key --> kept secret by CA, used to SIGN certificates
21 # ca_public_key  --> shared with all vehicles, used to VERIFY
    signatures

```

Listing 11.1: CA key generation — annotated snippet

11.3 Full Claude Code Prompt for Step 2

Claude Code Prompt — CA Module

You are an expert Python security engineer. Create three production-ready Python modules for a Vehicle-to-Vehicle (V2V) security simulation's Certificate Authority.

REQUIREMENTS:

1. `ca.py` - Certificate Authority:
 - Generate ECDSA P-256 (SECP256R1) key pair for the CA
 - Create a self-signed X.509 v3 root certificate with:
 - * Subject: CN=V2V-Root-CA, O=V2V-Security-Lab
 - * Validity: 10 years from now
 - * Key usage: Certificate Sign, CRL Sign
 - * Basic constraints: CA=True, pathLen=1
 - Save `ca_key.pem` (private, PEM, encrypted with passphrase) and `ca_cert.pem` (public, PEM)
 - Print certificate details in human-readable form to stdout
 - Every function must have a full docstring with Args/Returns/Raises
2. `issue_cert.py` - Vehicle Certificate Issuer:
 - Accept CLI args: `--vehicle-id`, `--output-dir`
 - Generate ECDSA P-256 key pair for the vehicle

```

- Create Certificate Signing Request (CSR) for the vehicle
- Sign the CSR with the CA key to produce a vehicle
certificate:
  * Subject: CN={vehicle-id}, O=V2V-Security-Lab
  * Validity: 1 year from now
  * Key usage: Digital Signature
  * Extended Key usage: clientAuth
  * SAN: DNS:{vehicle-id}
- Save {vehicle_id}_cert.pem and {vehicle_id}_key.pem
- Display certificate fingerprint and serial number

3. verify_cert.py - Certificate Verifier:
  - Accept CLI args: --cert (path to cert to verify), --ca-cert
  - Verify the certificate's signature against the CA
  certificate
  - Check validity dates (not before/not after)
  - Display: VALID/INVALID, expiry date, subject, issuer, key
  type

Use ONLY the 'cryptography' (PyCA) library.
Use Python 3.10+ type hints throughout.
Include try/except with specific exception types from
  cryptography.exceptions.
Add logging with timestamps using the logging module.
Include a __main__ block with usage examples.
After each file, show the expected terminal output.

```

11.4 Expected Output

```

[2024-01-15 10:23:01] INFO: Generating CA key pair (ECDSA P-256)...
[2024-01-15 10:23:01] INFO: CA key pair generated successfully.
[2024-01-15 10:23:01] INFO: Creating self-signed root certificate...
[2024-01-15 10:23:01] INFO: Certificate created. Details:
  Subject:   CN=V2V-Root-CA, O=V2V-Security-Lab
  Issuer:    CN=V2V-Root-CA, O=V2V-Security-Lab (self-signed)
  Serial:    0x3A7B2C1D4E5F6A0B
  Valid from: 2024-01-15 10:23:01 UTC
  Valid to:   2034-01-15 10:23:01 UTC
  Key type:   ECDSA P-256
  Signature:  SHA256withECDSA
[2024-01-15 10:23:01] INFO: Saved: ca/certs/ca_cert.pem
[2024-01-15 10:23:01] INFO: Saved: ca/certs/ca_key.pem (ENCRYPTED)
[2024-01-15 10:23:01] SUCCESS: CA initialised. Share ca_cert.pem with all vehicles.

```

Listing 11.2: Expected output from ca.py

Chapter 12

Step 3 — Cryptographic Module

12.1 Theory: Why AES-GCM + ECDH + ECDSA Together?

These three algorithms cover distinct security needs:

Table 12.1: Cryptographic Algorithm Selection Rationale

Algorithm	Type	Why chosen	Standard
AES-256-GCM	Sym. Encryption	AEAD: encrypts AND authenticates; hardware accelerated	NIST FIPS 197
ECDH P-256	Key Exchange	Small keys; fast; provides forward secrecy	NIST SP 800-56A
ECDSA P-256	Digital Sig	Non-repudiation; small signatures; fast	FIPS 186-4
HMAC-SHA256	MAC	Fast message authentication for BSM integrity	FIPS 198-1
SHA-256	Hash	Standard; collision resistant; fast	FIPS 180-4

12.2 Small Code Segment: AES-GCM Encrypt/Decrypt

```
1 import os
2 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
3
```

```

4 def aes_gcm_encrypt(key: bytes, plaintext: bytes) -> tuple[bytes,
    bytes, bytes]:
5     """Encrypt plaintext with AES-256-GCM.
6
7     Args:
8         key: 32-byte (256-bit) symmetric key
9         plaintext: Any bytes to encrypt
10
11     Returns:
12         Tuple of (ciphertext, nonce, tag)
13         - ciphertext: encrypted data (same length as plaintext)
14         - nonce: 12-byte random value (MUST be unique per key)
15         - tag: 16-byte authentication tag (verifies integrity)
16     """
17     # Generate a cryptographically random 96-bit (12-byte) nonce
18     # os.urandom() uses the OS's CSPRNG (e.g., /dev/urandom on
    Linux)
19     # NEVER reuse a nonce with the same key --- catastrophic if you
    do!
20     nonce = os.urandom(12)
21
22     # Create an AESGCM cipher object with our 256-bit key
23     # AESGCM automatically validates key length (must be 16, 24, or
    32 bytes)
24     aesgcm = AESGCM(key)
25
26     # Encrypt: returns ciphertext WITH the 16-byte auth tag
    appended
27     # The tag is the last 16 bytes of the returned value
28     encrypted = aesgcm.encrypt(
29         nonce=nonce,          # Unique per message
30         data=plaintext,       # Data to encrypt
31         aad=None              # Additional Authenticated Data (not
    encrypted
32     )                          # but integrity-protected); None here
33
34     # Split the result: ciphertext is everything except last 16
    bytes
35     ciphertext = encrypted[:-16]
36     tag = encrypted[-16:]     # 16-byte Galois/Counter Mode auth
    tag
37
38     return ciphertext, nonce, tag

```

Listing 12.1: AES-256-GCM encrypt/decrypt — annotated

12.3 Full Claude Code Prompt for Step 3

Claude Code Prompt — crypto.py

```
Create a production-ready Python module 'crypto.py' for V2V
communication security.

IMPLEMENT these exact functions with full docstrings, type hints
, and error handling:

1. aes_gcm_encrypt(key: bytes, plaintext: bytes,
                   aad: bytes | None = None) -> tuple[bytes,
                   bytes, bytes]
Returns: (ciphertext, nonce, tag)
- Use AESGCM from cryptography.hazmat.primitives.ciphers.aead
- Generate nonce with os.urandom(12)
- Validate key is exactly 32 bytes; raise ValueError
  otherwise

2. aes_gcm_decrypt(key: bytes, ciphertext: bytes, nonce: bytes,
                   tag: bytes, aad: bytes | None = None) ->
  bytes
- Reassemble ciphertext+tag and call AESGCM.decrypt()
- Catch InvalidTag exception and raise a custom
  DecryptionError
- Return plaintext bytes

3. ecdh_key_exchange(private_key: EllipticCurvePrivateKey,
                      peer_public_key: EllipticCurvePublicKey) ->
  bytes
- Use ECDH from cryptography.hazmat.primitives.asymmetric.ec
- Returns raw shared secret bytes (32 bytes for P-256)

4. ecdsa_sign(private_key: EllipticCurvePrivateKey,
               message: bytes) -> bytes
- Hash message with SHA-256 (use Prehashed or hash-then-sign
  pattern)
- Use ECDSA with SHA256 from cryptography
- Return DER-encoded signature bytes

5. ecdsa_verify(public_key: EllipticCurvePublicKey,
                 message: bytes, signature: bytes) -> bool
- Verify DER signature; return False (do NOT raise) on
  invalid

6. hmac_compute(key: bytes, message: bytes) -> bytes
```

```

    - Use HMAC with SHA-256 from cryptography.hazmat.primitives.
      hmac
    - Return 32-byte HMAC digest

7. hmac_verify(key: bytes, message: bytes, expected_hmac: bytes)
    -> bool
    - Use hmac.compare_digest() for constant-time comparison
    - Return bool

8. derive_session_key(shared_secret: bytes,
                      nonce_a: bytes, nonce_b: bytes) -> bytes
    - Use HKDF with SHA-256
    - info = b"v2v-session-key-v1"
    - salt = nonce_a + nonce_b
    - Return 32-byte derived key

9. generate_ecdh_keypair() -> tuple[EllipticCurvePrivateKey,
                                   EllipticCurvePublicKey]
    - Generate fresh ephemeral P-256 key pair for ECDHE

10. serialize_public_key(pub_key) -> bytes
     serialize_private_key(priv_key, password: bytes | None =
None) -> bytes
     load_public_key(data: bytes) -> EllipticCurvePublicKey
     load_private_key(data: bytes, password: bytes | None = None)

Define custom exceptions: CryptoError, DecryptionError,
    SignatureError.

Add a demonstration __main__ block that:
    - Generates two key pairs (Alice and Bob)
    - Performs ECDH and derives session key
    - Encrypts a test message from Alice, decrypts at Bob's side
    - Signs a message, verifies signature
    - Shows HMAC compute and verify
    - Prints SUCCESS or FAILURE for each step with timing in ms

```

Chapter 13

Step 4 — Authentication Protocol

13.1 Theory: The Handshake in Detail

The four-step handshake (see Figure 6.1) achieves the following security goals:

1. **Step 1 (HELLO):** Vehicle A announces itself. It sends its certificate (so B can verify A's identity) and a nonce (random value) to prevent replay attacks.
2. **Step 2 (CHALLENGE):** Vehicle B verifies A's certificate. B sends its own certificate and a challenge: it signs both nonces together. This proves B has the private key matching the public key in its certificate.
3. **Step 3 (RESPONSE):** Vehicle A verifies B's certificate and B's signature. A signs the combined nonces back, proving it has its private key. A also sends its ECDH ephemeral public key to begin key exchange.
4. **Step 4 (ACK):** Vehicle B verifies A's signature and response. B sends its ECDH ephemeral public key. Both sides now compute the same shared secret and derive the session key using HKDF.

13.2 Small Code Segment: HELLO Message

```
1 import os
2 import json
3 import time
4 from dataclasses import dataclass, asdict
5
6 @dataclass
7 class HelloMessage:
8     """Step 1 of the V2V authentication handshake.
9
10     Sent by the initiating vehicle (Vehicle A) to begin
11     authentication.
12
13     Contains the vehicle's identity certificate and a fresh nonce.
```

```

12     """
13     vehicle_id: str          # Human-readable vehicle identifier
14     certificate_pem: str     # Vehicle's X.509 certificate in PEM
                               # format
15     nonce: str              # Random 32-byte value (hex-encoded
                               # string)
16     timestamp: float        # Unix timestamp: when this message was
                               # created
17     message_type: str = "HELLO" # Constant type identifier
18
19 def build_hello(vehicle_id: str, cert_pem: bytes) -> HelloMessage:
20     """Construct a HELLO message for the authentication handshake.
21     """
22     return HelloMessage(
23         vehicle_id=vehicle_id,
24         certificate_pem=cert_pem.decode('utf-8'), # PEM is text
25         nonce=os.urandom(32).hex(), # 32 random bytes as hex
26         timestamp=time.time() # Seconds since Unix epoch
27         # Receiver will reject if abs(now - timestamp) > 5 seconds
28     )
29
30 def serialize_hello(msg: HelloMessage) -> bytes:
31     """Serialize HELLO message to JSON bytes for transmission."""
32     return json.dumps(asdict(msg)).encode('utf-8')
33     # asdict() converts dataclass to dictionary
34     # json.dumps() converts dict to JSON string
35     # encode('utf-8') converts string to bytes for socket
    transmission

```

Listing 13.1: HELLO message construction — annotated

13.3 Full Claude Code Prompt for Step 4

Claude Code Prompt — auth_protocol.py

```

Create 'auth_protocol.py': the complete V2V mutual
authentication protocol.

IMPLEMENT these classes and functions:

DATACLASSES (with JSON serialization via to_json()/from_json()):
1. HelloMessage: vehicle_id, certificate_pem, nonce_hex,
    timestamp, msg_type="HELLO"

```

```

2. ChallengeMessage: vehicle_id, certificate_pem, nonce_hex,
    signed_nonces_hex,
                        timestamp, msg_type="CHALLENGE"
3. ResponseMessage: signed_nonces_hex, ecdh_public_key_pem,
    timestamp,
                        msg_type="RESPONSE"
4. AckMessage: ecdh_public_key_pem, session_established=True,
    msg_type="ACK"

CLASSES:

class ReplayCache:
    """Prevents replay attacks by tracking seen nonces."""
    def __init__(self, window_seconds: int = 300)
    def add_nonce(self, nonce: str) -> None # Raises
    ReplayError if seen
    def is_fresh(self, nonce: str) -> bool
    def cleanup_expired(self) -> int # Removes expired nonces,
    returns count

class TimestampValidator:
    """Rejects messages with timestamps outside acceptable
    window."""
    def __init__(self, max_age_seconds: float = 5.0)
    def validate(self, timestamp: float) -> None # Raises
    StaleMessageError

class AuthProtocol:
    """Implements the four-step V2V authentication handshake."""

    def __init__(self, vehicle_id, private_key, certificate,
    ca_cert)

    def build_hello(self) -> HelloMessage
    # Create HelloMessage; store our nonce for later
    verification

    def process_hello(self, msg: HelloMessage) ->
    ChallengeMessage
    # 1. Verify msg.certificate against CA cert (signature check
    )
    # 2. Validate timestamp with TimestampValidator
    # 3. Check nonce with ReplayCache
    # 4. Sign (their_nonce + our_nonce) with our private key
    # 5. Return ChallengeMessage

```

```

    def process_challenge(self, msg: ChallengeMessage) ->
    ResponseMessage
    # 1. Verify msg.certificate against CA cert
    # 2. Validate timestamp, check nonce
    # 3. Verify signed_nonces covers our nonce (from HELLO we
    sent)
    # 4. Generate ephemeral ECDH key pair
    # 5. Sign (our_nonce + their_nonce) with our private key
    # 6. Return ResponseMessage

    def process_response(self, msg: ResponseMessage,
                        hello_msg: HelloMessage) -> AckMessage
    # 1. Validate timestamp
    # 2. Verify signature on (initiator_nonce + our_nonce)
    # 3. Compute ECDH shared secret with their ephemeral key
    # 4. Derive session key: HKDF(shared_secret, nonce_a+nonce_b
    )
    # 5. Store session_key; return AckMessage with our ECDH
    pubkey

    def process_ack(self, msg: AckMessage) -> bytes
    # 1. Compute ECDH shared secret with their ephemeral key
    # 2. Derive same session key
    # 3. Return session_key bytes

    @property
    def session_key(self) -> bytes | None

Custom exceptions: AuthError, ReplayError, StaleMessageError,
    CertificateError
Logging: every step logged at DEBUG/INFO level with vehicle IDs
    and timestamps
Include a demo __main__ block simulating the full handshake
    between two instances.

```


Chapter 14

Step 5 — V2V Message Protocol

14.1 Theory: Basic Safety Messages (BSM)

Key Concept

SAE J2735 defines the *Basic Safety Message (BSM)* as the fundamental V2V data packet. A BSM contains:

- Vehicle ID (pseudonymous, 4 bytes)
- GPS position: latitude, longitude (WGS84, 1/10th micro-degree)
- Speed (0.02 m/s units)
- Heading (0.0125 degree units, 0–359.9875)
- Acceleration (longitudinal and lateral)
- Brake system status (all wheels)
- Vehicle size (width + length)

BSMs are broadcast 10 times per second. Our simulation sends them every second for clarity.

Key Concept

Why Sign-then-Encrypt? We always sign BEFORE encrypting:

1. **Sign:** Attach ECDSA signature to plaintext. This proves the *original sender's* identity (signing is tied to the private key).
2. **Encrypt:** Wrap the signed message in AES-GCM encryption. Now an attacker cannot even read the message, let alone forge the signature.

The alternative (Encrypt-then-Sign) has subtle vulnerabilities: an attacker could strip the ciphertext from one message and attach a different sender's signature, creating ambiguity about who signed what.

14.2 Small Code Segment: BSM Structure

```

1 import time
2 from dataclasses import dataclass, field
3 from typing import Optional
4
5 @dataclass
6 class BasicSafetyMessage:
7     """SAE J2735-inspired Basic Safety Message for V2V
8     communication.
9
10    Transmitted 10x/second in real V2V; every second in our
11    simulation.
12    All fields except vehicle_id and timestamp are observable
13    vehicle state.
14    """
15    vehicle_id: str          # Pseudonymous ID (not real plate
16                             # number)
17    speed_kmh: float         # Speed in km/h (0.0 to 300.0)
18    heading_deg: float       # Compass heading in degrees (0.0 =
19                             # North)
20    latitude: float          # GPS latitude (decimal degrees, WGS84
21                             # )
22    longitude: float         # GPS longitude (decimal degrees,
23                             # WGS84)
24    timestamp: float = field( # Auto-filled if not provided
25                             # default_factory=time.time)
26    sequence_num: int = 0     # Incremented per message; detects
27                             # missing msgs
28    brake_applied: bool = False # True if brakes engaged
29    acceleration: float = 0.0  # m/s^2 (positive=accelerating)
30
31    def to_bytes(self) -> bytes:
32        """Serialize BSM to JSON bytes for signing and transmission.
33        """
34        import json
35        return json.dumps({
36            'vehicle_id': self.vehicle_id,
37            'speed_kmh': self.speed_kmh,
38            'heading_deg': self.heading_deg,
39            'latitude': self.latitude,
40            'longitude': self.longitude,
41            'timestamp': self.timestamp, # Used for replay
42            'sequence_num': self.sequence_num, # Used for gap
43            'brake_applied': self.brake_applied,

```

```

35         'acceleration': self.acceleration,
36     }).encode('utf-8')
37     # encode() converts string to bytes (required for crypto
    operations)

```

Listing 14.1: BasicSafetyMessage dataclass — annotated

14.3 Full Claude Code Prompt for Step 5

Claude Code Prompt — v2v_protocol.py

```

Create 'v2v_protocol.py': the complete V2V secure message
protocol.

IMPLEMENT:

@dataclass
class BasicSafetyMessage:
    vehicle_id: str
    speed_kmh: float
    heading_deg: float
    latitude: float
    longitude: float
    timestamp: float = field(default_factory=time.time)
    sequence_num: int = 0
    brake_applied: bool = False
    acceleration_ms2: float = 0.0
    Methods: to_bytes(), from_bytes(data: bytes), to_dict(),
    from_dict(d: dict)

@dataclass
class SecureBSM:
    """Wrapper adding cryptographic security fields to a BSM."""
    bsm_ciphertext: bytes    # AES-GCM encrypted BSM
    nonce: bytes             # 12-byte AES-GCM nonce
    auth_tag: bytes          # 16-byte AES-GCM auth tag
    signature: bytes         # ECDSA signature over plaintext
    BSM
    sender_id: str           # Vehicle ID (for quick lookup)
    sequence_num: int        # Copy of inner seq num for replay
    check
    Methods: to_bytes(), from_bytes()

class SequenceNumberManager:
    """Tracks and validates message sequence numbers per peer

```

```

    """
    def __init__(self, max_gap: int = 10)
    def validate(self, sender_id: str, seq_num: int) -> bool
    # Returns False if: seq_num already seen, or gap > max_gap
    def get_stats(self, sender_id: str) -> dict
    # Returns: {received, missing, last_seq, gap_count}

class MessageLogger:
    """Logs all sent/received messages with security properties
    """
    def log_sent(self, bsm: BasicSafetyMessage, secure:
    SecureBSM)
    def log_received(self, secure: SecureBSM, bsm:
    BasicSafetyMessage,
                    security_props: dict)
    # security_props keys: authenticated, encrypted,
    integrity_verified,
    #                                replay_protected, signature_valid
    def get_recent(self, count: int = 20) -> list[dict]
    def get_alerts(self) -> list[dict]

FUNCTIONS:
def secure_send(bsm: BasicSafetyMessage, session_key: bytes,
                signing_key, seq_mgr: SequenceNumberManager,
                logger: MessageLogger) -> bytes:
    """Sign-then-encrypt a BSM for transmission.
    1. Serialize BSM to bytes
    2. ECDSA sign the serialized bytes (produces signature)
    3. AES-GCM encrypt(BSM_bytes + signature) with session_key
    4. Build SecureBSM wrapper
    5. Log and return serialized SecureBSM bytes
    """

def secure_receive(data: bytes, session_key: bytes,
                  peer_verify_key, seq_mgr:
    SequenceNumberManager,
                  replay_cache, logger: MessageLogger,
                  timestamp_validator) -> BasicSafetyMessage:
    """Decrypt, verify, and parse a received SecureBSM.
    1. Parse SecureBSM from bytes
    2. Check sequence number (replay/gap detection)
    3. AES-GCM decrypt ciphertext using session_key
    4. Split decrypted bytes into BSM_bytes and signature
    5. ECDSA verify signature against peer_verify_key
    6. Check BSM timestamp with timestamp_validator

```

```
7. Parse and return BasicSafetyMessage
8. Log with full security_props dict
On failure: raise specific exception (TamperError,
ReplayError, etc.)
"""
```

```
Custom exceptions: TamperError, ReplayError, DecryptionError,
SequenceError
```

Chapter 15

Step 6 — Main Vehicle Node

15.1 Theory: Socket Programming and Threading

Key Concept

TCP Socket programming basics:

- A **socket** is one end of a network connection — like a phone handset. Two sockets connected together form a communication channel.
- **Server socket**: calls `bind()` (sets up address/port), `listen()` (waits for calls), `accept()` (answers a call).
- **Client socket**: calls `connect()` to dial the server.
- **Threading**: We need a separate thread for each connection because `accept()` blocks until a new connection arrives — if we did not use threads, we could only handle one vehicle at a time.

15.2 Small Code Segment: Socket Server

```
1 import socket
2 import threading
3 import logging
4
5 class VehicleNode:
6     def __init__(self, vehicle_id: str, port: int):
7         self.vehicle_id = vehicle_id
8         self.port = port
9         self.running = False          # Flag to control main loop
10
11         # AF_INET = IPv4, SOCK_STREAM = TCP (reliable, ordered)
12         self.server_sock = socket.socket(
13             socket.AF_INET, socket.SOCK_STREAM
14         )
```

```

15
16     # Allow port reuse immediately after process ends
17     # Without this, you often get "Address already in use"
errors
18     # when restarting the node during development
19     self.server_sock.setsockopt(
20         socket.SOL_SOCKET, # Socket-level option
21         socket.SO_REUSEADDR, # Option: allow address reuse
22         1 # Value: enable (1 = True)
23     )
24
25     def start_server(self):
26         """Bind to port and begin accepting connections in
background."""
27         # Bind to all interfaces (0.0.0.0) on our port
28         # '' = '0.0.0.0' = listen on all network interfaces
29         self.server_sock.bind(('', self.port))
30
31         # Listen with a backlog of 5 pending connections
32         # (queue of callers waiting to be accepted)
33         self.server_sock.listen(5)
34         logging.info(f"[{self.vehicle_id}] Listening on port {self.
port}")
35         self.running = True
36
37         # Start the accept loop in a daemon thread
38         # daemon=True means thread dies when main program exits
39         t = threading.Thread(target=self._accept_loop, daemon=True)
40         t.start()
41
42     def _accept_loop(self):
43         """Continuously accept incoming connections."""
44         while self.running:
45             try:
46                 conn, addr = self.server_sock.accept() # Blocks
here
47
48                 # Start a new thread for each connection
49                 ct = threading.Thread(
50                     target=self.handle_connection,
51                     args=(conn, addr),
52                     daemon=True
53                 )
54                 ct.start()
55             except OSError:
56                 break # Socket was closed --- exit loop

```

Listing 15.1: TCP socket server setup — annotated

15.3 Full Claude Code Prompt for Step 6

Claude Code Prompt — vehicle_node.py

```
Create 'vehicle_node.py': the complete V2V vehicle node
implementation.

class VehicleNode:
    def __init__(self, vehicle_id: str, port: int,
                  cert_path: str, key_path: str, ca_cert_path:
str,
                  bsm_interval_seconds: float = 1.0)
        # Loads certificate and private key from PEM files
        # Initialises: auth_protocol, seq_manager, replay_cache,
        #               timestamp_validator, message_logger,
        flask_state dict

    def start_server(self) -> None
        # Bind TCP socket to self.port
        # Start _accept_loop in daemon thread
        # Log: "[vehicle_id] Listening on port {port}"

    def connect_to_peer(self, host: str, port: int) -> bool
        # TCP connect to (host, port)
        # Run full authentication handshake as INITIATOR:
        #   build_hello -> send -> recv challenge ->
process_challenge
        #   -> send response -> recv ack -> process_ack
        # Store peer's public key and session_key
        # Start receive_loop and bsm_send_loop in threads
        # Return True on success, False on failure
        # Log each handshake step with timing

    def handle_connection(self, conn: socket.socket, addr: tuple
) -> None
        # Run full authentication handshake as RESPONDER:
        #   recv hello -> process_hello -> send challenge
        #   -> recv response -> process_response -> send ack
        # Store peer info and session_key
        # Start receive_loop and bsm_send_loop

    def _bsm_send_loop(self, conn: socket.socket) -> None
        # Every self.bsm_interval_seconds seconds:
        #   Build BSM with simulated speed/heading/GPS (slightly
random)
        #   Call secure_send() and transmit bytes over conn
```



```

# Log: "Sent BSM #N: speed=X, heading=Y"
# Increment sequence number

def _receive_loop(self, conn: socket.socket) -> None
# Loop: recv bytes, call secure_receive()
# Log received BSM content and all security properties
# On TamperError/ReplayError: log ALERT and add to
flask_state alerts

def get_state(self) -> dict
# Return dict for Flask dashboard:
# {vehicle_id, cert_status, peer_id, auth_status,
session_active,
# recent_messages: [...], alerts: [...], stats: {...}}

def shutdown(self) -> None
# Set self.running = False
# Close all sockets
# Log: "Vehicle node shutdown complete"

CLI arguments (argparse):
--vehicle-id (required)
--port (required)
--cert path to certificate PEM
--key path to private key PEM
--ca-cert path to CA certificate PEM
--peer-host host to connect to (optional; if omitted, wait
for connection)
--peer-port port to connect to
--bsm-interval seconds between BSMs (default: 1.0)
--dashboard-port Flask port (default: 5000)

Start Flask dashboard in a daemon thread.
Use structured logging with vehicle_id prefix on every message.
Handle KeyboardInterrupt gracefully with shutdown().

```

Chapter 16

Step 7 — Flask Dashboard

16.1 Theory: Flask REST API

Key Concept

Flask is a micro web framework for Python. Our dashboard works like this:

1. The vehicle node runs a Flask server in a background thread
2. A browser opens <http://localhost:5000>
3. The browser periodically fetches `/api/state` — a JSON endpoint that returns current vehicle status
4. The HTML page uses the browser's `fetch()` API to call this endpoint every 3 seconds and update the display

This avoids WebSockets or complex frameworks — just plain HTTP polling.

16.2 Full Claude Code Prompt for Step 7

Claude Code Prompt — Flask Dashboard

```
Create 'dashboard/app.py' and 'dashboard/templates/dashboard.html'
for the V2V security Flask dashboard.

app.py requirements:
- Flask app with CORS headers (for dev convenience)
- Route GET / -> render dashboard.html
- Route GET /api/state -> return node.get_state() as JSON
- Route GET /api/alerts -> return node.get_state()['alerts'] as
  JSON
- Route POST /api/clear_alerts -> clear the alerts list
- Accept 'node' as parameter (VehicleNode instance) to embed in
  app context
```

```

- Run on configurable port (default 5000)

dashboard.html requirements (NO JavaScript frameworks, plain
  HTML/CSS/fetch only):
- Full-page layout with dark header: "V2V Security Dashboard - {
  vehicle_id}"
- CSS: grid layout, green/red status badges, blinking red for
  alerts
- Three status panels across top:
  Panel 1 "Certificate Status":
    - Badge: GREEN "VALID" or RED "EXPIRED" or YELLOW "WARNING"
    - Certificate CN, issuer, expiry date
  Panel 2 "Authentication Status":
    - Badge: GREEN "AUTHENTICATED" or RED "NOT AUTH" or ORANGE
    "PENDING"
    - Peer vehicle ID, session key status, forward secrecy
    flag
  Panel 3 "Security Alerts":
    - Count of active alerts
    - Latest alert text (or "No alerts" if none)
    - "Clear Alerts" button

- Messages table (last 20, newest first):
  Columns: Time | From/To | Speed | Heading | Auth | Enc | Int
  | Rply
  Last four columns show green checkmark or red X
  Row background: yellow if any security property is false

- Auto-refresh every 3 seconds using:
  setInterval(async () => {
    const r = await fetch('/api/state');
    const data = await r.json();
    updateUI(data);
  }, 3000);

- If alerts array is non-empty, make the alerts panel blink red
  (CSS animation)

Styling: minimal, professional, dark header (#1a237e), white
  body, monospace font
  for IDs and hex values. Mobile-responsive. No external CSS
  frameworks.

```

Chapter 17

Step 8 — Attack Demonstration Scripts

17.1 8a — Replay Attack

Theory: The attacker:

1. Captures a valid, encrypted BSM from Vehicle A at time T
2. Waits 10 seconds (beyond the 5-second replay window)
3. Retransmits the *identical bytes* to Vehicle B

Detection: Vehicle B's replay cache has stored the nonce from the original message. When the replay arrives, `ReplayCache.add_nonce()` raises `ReplayError` because the nonce is already in the cache. The timestamp check also catches it: $\text{now} - T > 5\text{s}$.

Claude Code Prompt — `replay_attack.py`

Create `'attacks/replay_attack.py'`:

A script that:

1. Connects to Vehicle A's TCP port and INTERCEPTS one BSM (acts as a proxy: receives from A, logs the raw bytes, forwards to B)
2. Waits `self.delay` seconds (default: 10, larger than 5-second replay window)
3. Opens a NEW connection to Vehicle B and sends the SAME raw bytes again
4. Captures Vehicle B's response (which should be an error/rejection)
5. Prints a coloured terminal report:
 - ATTACK SENT: timestamp, size of replayed packet
 - RESULT: REJECTED (if B logs replay error) or ACCEPTED (if attack succeeded)
 - Reason shown: "Nonce already seen" or "Timestamp too old"

```

The script should NOT authenticate --- it just reads raw bytes
off the wire.
Use argparse: --vehicle-a-port, --vehicle-b-host, --vehicle-b-
port, --delay
Print ASCII art banner: "[ REPLAY ATTACK DEMONSTRATION ]"
Expected output must show the attack being REJECTED.
Add comments explaining exactly what makes this attack fail.

```

```

+=====+
|  [ REPLAY ATTACK DEMONSTRATION ]  |
+=====+
[10:45:01] Captured BSM from Vehicle A (312 bytes)
[10:45:01] BSM nonce: a3f2e1d09b8c7a65...
[10:45:01] BSM timestamp: 10:45:01.234
[10:45:01] Waiting 10 seconds before replaying...
[10:45:11] Sending captured BSM to Vehicle B...
[10:45:11] Vehicle B response: ERROR

RESULT: ATTACK REJECTED
Reason: Timestamp too old (10.1s > 5.0s window)
        AND Nonce a3f2e1d09b8c7a65 already in replay cache

SECURITY PROPERTIES THAT PREVENTED THIS ATTACK:
  [[OK]] Timestamp validation (5-second replay window)
  [[OK]] Nonce cache (256-bit nonces stored for 300 seconds)
  [[OK]] Sequence number gap detected (seq #7 seen twice)

```

Listing 17.1: Expected replay attack output

17.2 8b — Tamper Attack

Theory: The attacker sits between A and B (by ARP spoofing or network reconfiguration). It intercepts encrypted BSMs, modifies bytes in the ciphertext, and forwards the modified bytes. Because AES-GCM provides authenticated encryption, any modification to the ciphertext changes the authentication tag — the receiver’s decryption attempt fails.

Claude Code Prompt — `tamper_attack.py`

```

Create 'attacks/tamper_attack.py':

A proxy script that:
1. Listens on a port between A and B (e.g., 9003)
2. Forwards bytes FROM A TO B, but flips 3 random bits in the
   ciphertext

```

```

3. Logs: original bytes (hex), modified bytes (hex), which bits
   were flipped
4. Captures B's response

Expected result: B's AES-GCM decryption raises InvalidTag error
(the authentication tag no longer matches the modified
   ciphertext)

Print banner: "[ TAMPER ATTACK DEMONSTRATION ]"
Show exactly which bytes were modified and their before/after
   values.
Show the error Vehicle B raises.
Explain in output why the attack was caught.

argparse: --listen-port, --vehicle-b-host, --vehicle-b-port, --
   flip-bytes (default: 3)

```

```

+=====+
|  [ TAMPER ATTACK DEMONSTRATION ]  |
+=====+
[10:46:03] Intercepted packet (312 bytes) from Vehicle A
[10:46:03] MODIFYING bytes: positions [87, 143, 201]
   Byte[87]:  0xA3 -> 0xB3  (bit 4 flipped)
   Byte[143]: 0x2F -> 0x2E  (bit 0 flipped)
   Byte[201]: 0xC8 -> 0xC0  (bit 3 flipped)
[10:46:03] Forwarding MODIFIED packet to Vehicle B...
[10:46:03] Vehicle B raised: InvalidTag - AES-GCM authentication FAILED

RESULT: ATTACK DETECTED AND REJECTED
Reason: AES-256-GCM authentication tag mismatch
   Even 1-bit change in ciphertext produces different tag

SECURITY PROPERTIES THAT PREVENTED THIS ATTACK:
  [[OK]] AES-256-GCM Authenticated Encryption (detects any modification)
  [[OK]] ECDSA signature (would also fail on decrypted plaintext)
  [[OK]] HMAC-SHA256 (additional integrity layer)

```

Listing 17.2: Expected tamper attack output

17.3 8c — Man-in-the-Middle Attack

Claude Code Prompt — mitm_attack.py

```

Create 'attacks/mitm_attack.py':

Simulate a MITM attacker that:
1. Intercepts Vehicle A's HELLO message during authentication
2. Attempts to replace Vehicle A's certificate with a FAKE self-

```

```

    signed certificate
    that the attacker generates on the fly
3. Forwards the modified HELLO (with fake cert) to Vehicle B

Expected result: Vehicle B verifies the certificate's signature
    against the CA
    certificate. Since the fake cert was NOT signed by the real CA,
    verification FAILS.
Raise: CertificateError: "Certificate signature verification
    failed"

Additionally:
4. Show what happens if the attacker simply RELAYS the HELLO
    unchanged:
    The attacker cannot decrypt the subsequent messages because
    they do not have
    Vehicle A's private key. Show that the challenge-response
    proves this.

Print banner: "[ MAN-IN-THE-MIDDLE ATTACK DEMONSTRATION ]"
Show TWO scenarios:
    A) Attacker replaces certificate -> CA verification fails
    B) Attacker relays everything -> Cannot forge challenge
        response
        (would need Vehicle A's private key)

argparse: --listen-port, --vehicle-b-host, --vehicle-b-port

```

```

+=====+
|  [ MAN-IN-THE-MIDDLE ATTACK DEMO ]  |
+=====+
SCENARIO A: Certificate Replacement Attack
[10:47:01] Intercepted HELLO from Vehicle A
[10:47:01] Generating fake certificate (self-signed)...
[10:47:01] Replacing cert in HELLO with fake cert
[10:47:01] Forwarding modified HELLO to Vehicle B
[10:47:01] Vehicle B: Verifying certificate against CA...
[10:47:01] RESULT: CertificateError: CA signature verification FAILED
           Fake cert not signed by V2V-Root-CA

SCENARIO B: Relay Attack
[10:47:02] Relaying authentic HELLO unchanged...
[10:47:02] Vehicle B sends CHALLENGE with nonce: f3a2b1c0...
[10:47:02] Challenge requires: Sign(nonce_a + nonce_b) with Vehicle A's private key
[10:47:02] Attacker does NOT have Vehicle A's private key
[10:47:02] RESULT: Cannot forge RESPONSE -> Authentication fails

CONCLUSION:
  [[OK]] X.509 certificates tied to CA-verified identities
  [[OK]] Challenge-response proves private key ownership

```

```
[[OK]] Certificate replacement detected by CA signature check
```

Listing 17.3: Expected MitM attack output

Chapter 18

Step 9 — Testing Guide

18.1 Running the Full Demo

```
# =====
# STEP 0: Setup (do once)
# =====
cd v2v_security
source venv/bin/activate
python ca/ca.py                                # Generate CA
python ca/issue_cert.py --vehicle-id vehicle-a --output-dir ca/certs
python ca/issue_cert.py --vehicle-id vehicle-b --output-dir ca/certs

# =====
# STEP 1: Terminal 1 --- Start Vehicle A (server + dashboard)
# =====
python node/vehicle_node.py \
    --vehicle-id vehicle-a \
    --port 9001 \
    --cert ca/certs/vehicle_a_cert.pem \
    --key ca/certs/vehicle_a_key.pem \
    --ca-cert ca/certs/ca_cert.pem \
    --dashboard-port 5001

# =====
# STEP 2: Terminal 2 --- Start Vehicle B (connect to A)
# =====
python node/vehicle_node.py \
    --vehicle-id vehicle-b \
    --port 9002 \
    --cert ca/certs/vehicle_b_cert.pem \
    --key ca/certs/vehicle_b_key.pem \
    --ca-cert ca/certs/ca_cert.pem \
    --peer-host 127.0.0.1 \
    --peer-port 9001 \
```

```

--dashboard-port 5002

# =====
# STEP 3: View Dashboards in browser
# =====
# Vehicle A dashboard: http://localhost:5001
# Vehicle B dashboard: http://localhost:5002

# =====
# STEP 4: Terminal 3 --- Run attack demonstrations
# =====
python attacks/replay_attack.py \
    --vehicle-a-port 9001 \
    --vehicle-b-host 127.0.0.1 \
    --vehicle-b-port 9002 \
    --delay 10

python attacks/tamper_attack.py \
    --listen-port 9003 \
    --vehicle-b-host 127.0.0.1 \
    --vehicle-b-port 9002

python attacks/mitm_attack.py \
    --listen-port 9004 \
    --vehicle-b-host 127.0.0.1 \
    --vehicle-b-port 9002

```

Listing 18.1: Step-by-step demo commands

18.2 Expected Successful Authentication Output

```

[10:30:00.001] [vehicle-b] INFO: Connecting to peer at 127.0.0.1:9001
[10:30:00.012] [vehicle-b] INFO: TCP connection established
[10:30:00.013] [vehicle-b] INFO: Sending HELLO {nonce=a3f2..., ts=10:30:00.013}
[10:30:00.021] [vehicle-a] INFO: Received HELLO from vehicle-b
[10:30:00.022] [vehicle-a] INFO: Verifying vehicle-b certificate...
[10:30:00.025] [vehicle-a] INFO: Certificate VALID (CN=vehicle-b, exp=2025-01-01)
[10:30:00.026] [vehicle-a] INFO: Sending CHALLENGE {nonce=b4g3..., signed_nonces
    =...}
[10:30:00.034] [vehicle-b] INFO: Received CHALLENGE from vehicle-a
[10:30:00.035] [vehicle-b] INFO: Verifying vehicle-a certificate...
[10:30:00.038] [vehicle-b] INFO: Certificate VALID (CN=vehicle-a, exp=2025-01-01)
[10:30:00.039] [vehicle-b] INFO: Challenge signature verified OK
[10:30:00.040] [vehicle-b] INFO: Sending RESPONSE {ecdh_pubkey=...}
[10:30:00.048] [vehicle-a] INFO: Received RESPONSE from vehicle-b
[10:30:00.049] [vehicle-a] INFO: Response signature verified OK
[10:30:00.050] [vehicle-a] INFO: Computing ECDH shared secret...
[10:30:00.051] [vehicle-a] INFO: Deriving session key (HKDF-SHA256)...
[10:30:00.052] [vehicle-a] INFO: Sending ACK {ecdh_pubkey=...}
[10:30:00.060] [vehicle-b] INFO: Received ACK

```

```
[10:30:00.061] [vehicle-b] INFO: Computing ECDH shared secret...
[10:30:00.062] [vehicle-b] INFO: Session key derived. Forward secrecy: ENABLED
[10:30:00.063] [vehicle-b] SUCCESS: MUTUAL AUTHENTICATION COMPLETE (62ms)
[10:30:00.064] [vehicle-a] SUCCESS: MUTUAL AUTHENTICATION COMPLETE (62ms)
[10:30:01.001] [vehicle-a] INFO: BSM #1 sent: speed=72.3 km/h, heading=045deg
[10:30:01.002] [vehicle-b] INFO: BSM #1 received: AUTHENTICATED|ENCRYPTED|VERIFIED|
REPLAY-PROTECTED
```

Listing 18.2: Expected authentication log output

18.3 Evaluator Demo Script

1. **Start the CA** and show the generated certificate file in PEM format. Explain: “This is the trust anchor — both vehicles trust certificates signed by this CA.”
2. **Start Vehicle A and B.** Point to the log showing the four-step handshake with timings. Explain each step as it appears.
3. **Open both dashboards.** Show green authenticated status, real-time BSMs appearing in the table, all four security property checkmarks.
4. **Run the replay attack.** Show the REJECTED message in both the terminal and the flashing alert panel in the dashboard.
5. **Run the tamper attack.** Show the AES-GCM authentication tag failure.
6. **Run the MitM attack.** Show the CA signature verification failure.
7. **Run the evaluation script** (see Step 19) and display the security metrics table.

Chapter 19

Step 10 — Security Evaluation Metrics

19.1 Measurement Methodology

Table 19.1: Security Evaluation Metrics

Metric	How to Measure	Expected	Tool
Auth latency	<code>time.perf_counter()</code> around full handshake	<200ms	Python time
Encrypt throughput	Encrypt 10,000 BSMs, measure total time	>5000/s	timeit
Decrypt throughput	Decrypt 10,000 BSMs	>5000/s	timeit
Replay detect rate	Send 100 replayed msgs; count rejections	100%	replay_attack.py
Tamper detect rate	Send 100 tampered msgs; count rejections	100%	tamper_attack.py
Cert verify time	Verify 1000 certs; average time	<5ms	Python time
Nonce uniqueness	Generate nonces; check collisions	10 ⁶ 0 collisions	Python set
Forward secrecy	Check new keys generated each session	New keys each time	Logging

19.2 Full Claude Code Prompt — Evaluation Script

Claude Code Prompt — evaluate_security.py

```
Create 'tests/evaluate_security.py': a comprehensive security
evaluation script.

The script must:
1. Authentication Latency:
  - Run 100 authentication handshakes between two in-process
    VehicleNode instances
  - Record each handshake duration in milliseconds
  - Report: min, max, mean, median, p95, p99, std deviation

2. Crypto Throughput:
  - Generate 10,000 BSMs
  - Benchmark: aes_gcm_encrypt (ops/second and MB/s)
  - Benchmark: aes_gcm_decrypt (ops/second and MB/s)
  - Benchmark: ecdsa_sign (ops/second)
  - Benchmark: ecdsa_verify (ops/second)
  - Benchmark: hmac_compute (ops/second)

3. Replay Detection Rate:
  - Create 100 valid SecureBSMs with unique nonces
  - Attempt to replay each one after 6 seconds
  - Report: X/100 replays correctly rejected
  - Also test: 0-second replay (nonce cache hit)

4. Tamper Detection Rate:
  - Create 100 valid SecureBSMs
  - Flip 1, 2, and 5 random bits in each
  - Attempt to decrypt each
  - Report: X/100 tampered messages correctly detected per bit-
    flip level

5. Certificate Verification Time:
  - Verify the same certificate 1000 times
  - Report: mean and p99

6. Nonce Uniqueness:
  - Generate 1,000,000 nonces using os.urandom(32)
  - Check for any duplicates
  - Report: 0 collisions expected

7. Forward Secrecy Check:
  - Run 10 sessions
```

```
- Verify each session uses a DIFFERENT ephemeral ECDH key pair
- Verify session keys are all different

Output a formatted report table to stdout AND save as '
    security_report.txt'.
Include PASS/FAIL/WARN for each metric based on thresholds.
```

Chapter 20

Step 11 — MQTT Network Setup

20.1 Theory: MQTT vs TCP Sockets

Table 20.1: MQTT vs TCP Sockets for V2V Simulation

Aspect	TCP Sockets	MQTT
Connection model	Point-to-point (1-to-1)	Pub/sub (1-to-many)
Scalability	New connection per vehicle	One broker, many vehicles
Infrastructure	None needed	MQTT broker required
V2V realism	Low (V2V is broadcast)	Higher (V2V is broadcast)
Complexity	Simpler code	Slightly more setup
Use for	Two-vehicle tests	Multi-vehicle simulations

20.2 MQTT Setup

```
# Install Mosquitto (Linux)
sudo apt-get install mosquitto mosquitto-clients

# Start Mosquitto broker
mosquitto -v                                # verbose mode for debugging
# Default: listens on localhost:1883

# Test MQTT without our code:
# Terminal A: subscribe
mosquitto_sub -h localhost -t "v2v/#" -v

# Terminal B: publish test message
mosquitto_pub -h localhost -t "v2v/vehicle-a/bsm" \
    -m '{"test": "hello from Vehicle A"}'
```



```
# You should see the message appear in Terminal A instantly
```

Listing 20.1: Install and configure Mosquitto MQTT broker

20.3 Full Claude Code Prompt — MQTT Vehicle Node Variant

Claude Code Prompt — vehicle_node_mqtt.py

Create 'node/vehicle_node_mqtt.py': adapt vehicle_node.py to use MQTT.

Use paho-mqtt library. Implement class MQTTVehicleNode with same interface as VehicleNode but using MQTT for transport:

Topics:

Publish BSMs to:	v2v/{vehicle_id}/bsm
Subscribe to BSMs on:	v2v/+ /bsm (wildcard, all vehicles)
Auth messages:	v2v/{vehicle_id}/auth
Alerts:	v2v/alerts

Authentication via MQTT:

- Auth still happens in a separate TCP handshake (use the existing auth_protocol.py)
- After authentication establishes session_key, use MQTT for BSM exchange
- Auth handshake TCP port = MQTT broker port + 100

MQTT callbacks:

```
on_connect: subscribe to topics, log connection
on_message: deserialize, call secure_receive, log
on_disconnect: log, attempt reconnect with backoff
```

BSM publishing:

- Publish SecureBSM bytes as payload (binary)
- QoS level 1 (at least once delivery)
- retain=False (BSMs are ephemeral)

```
argparse: --broker-host (default localhost), --broker-port (
    default 1883),
    --vehicle-id, --cert, --key, --ca-cert
```

Chapter 21

Step 12 — Docker Setup

21.1 Theory: What Docker Provides

Key Concept

Docker packages an application and all its dependencies into a **container** — an isolated, reproducible environment. Benefits:

- Consistent behaviour across machines (“works on my machine” solved)
- Easy multi-node simulation on one laptop
- Reproducible for evaluators: `docker compose up` runs everything

Docker Compose orchestrates multiple containers, defining which containers exist, how they connect, and what environment variables they receive.

21.2 Dockerfile

```
1 # --- Dockerfile for V2V Vehicle Node ---
2 # syntax=docker/dockerfile:1
3
4 # Start from official Python 3.11 slim image
5 # "slim" = smaller image, only essential packages
6 FROM python:3.11-slim
7
8 # Set working directory inside the container
9 # All subsequent commands run from here
10 WORKDIR /app
11
12 # Copy requirements first (Docker layer caching optimisation)
13 # If requirements.txt hasn't changed, Docker reuses cached layer
14 # This avoids re-downloading packages on every code change
15 COPY requirements.txt .
16
```

```

17 # Install Python dependencies
18 # --no-cache-dir: don't store pip cache (keeps image smaller)
19 RUN pip install --no-cache-dir -r requirements.txt
20
21 # Copy the rest of the project code
22 # This layer changes every time code changes (as expected)
23 COPY . .
24
25 # The container will run as vehicle_node.py by default
26 # Arguments are passed via docker-compose.yml environment/command
27 ENTRYPOINT ["python", "node/vehicle_node.py"]
28
29 # Document which ports this container uses (documentation only)
30 # Actual port mapping is in docker-compose.yml
31 EXPOSE 9001 9002 5000

```

Listing 21.1: Dockerfile — annotated line by line

21.3 docker-compose.yml

```

1 # docker-compose.yml
2 version: '3.9'
3
4 # Define the shared network all containers use
5 networks:
6   v2v-net:
7     driver: bridge      # Standard Docker bridge network
8
9 services:
10  # --- Certificate Authority (runs first, issues certs to
11  #    vehicles) ---
12  ca-server:
13    build: .
14    entrypoint: ["python", "ca/ca.py", "--output-dir", "/certs"]
15    volumes:
16      - ./ca/certs:/certs      # Mount host certs folder into
17                                # container
18    networks: [v2v-net]
19
20  # --- Vehicle A ---
21  vehicle-a:
22    build: .
23    command:
24      - "--vehicle-id=vehicle-a"
25      - "--port=9001"
26      - "--cert=/certs/vehicle_a_cert.pem"

```

```

25     - "--key=/certs/vehicle_a_key.pem"
26     - "--ca-cert=/certs/ca_cert.pem"
27     - "--dashboard-port=5001"
28     volumes:
29         - ./ca/certs:/certs      # Shared certs volume
30     ports:
31         - "9001:9001"            # Vehicle A node port (host:container)
32         - "5001:5001"            # Vehicle A dashboard
33     networks: [v2v-net]
34     depends_on: [ca-server]
35
36     # ---- Vehicle B ----
37     vehicle-b:
38         build: .
39         command:
40             - "--vehicle-id=vehicle-b"
41             - "--port=9002"
42             - "--cert=/certs/vehicle_b_cert.pem"
43             - "--key=/certs/vehicle_b_key.pem"
44             - "--ca-cert=/certs/ca_cert.pem"
45             - "--peer-host=vehicle-a" # Use container name as hostname
46             - "--peer-port=9001"
47             - "--dashboard-port=5002"
48         volumes:
49             - ./ca/certs:/certs
50         ports:
51             - "9002:9002"
52             - "5002:5002"
53         networks: [v2v-net]
54         depends_on: [vehicle-a] # Wait for Vehicle A to start
55
56     # ---- Mosquitto MQTT Broker ----
57     mosquitto:
58         image: eclipse-mosquitto:2.0 # Official Mosquitto image
59         ports:
60             - "1883:1883"
61         networks: [v2v-net]
62         volumes:
63             - ./docker/mosquitto.conf:/mosquitto/config/mosquitto.conf

```

Listing 21.2: docker-compose.yml — multi-container V2V simulation

21.4 Docker Commands

```

# Build all images
docker compose build

```

```
# Start all containers in background
docker compose up -d

# View logs from all containers (Ctrl+C to stop following)
docker compose logs -f

# View logs from one container only
docker compose logs vehicle-a -f

# Check container status
docker compose ps

# Stop all containers
docker compose down

# Stop and remove volumes (clean slate)
docker compose down -v

# Open a shell inside a running container (for debugging)
docker compose exec vehicle-a /bin/bash
```

Listing 21.3: Docker build and run commands

Chapter 22

Step 13 — Two-Laptop Setup Guide

22.1 Network Configuration

```
# =====
# PREREQUISITE: Both laptops on same network (LAN or hotspot)
# =====

# --- On LAPTOP A (192.168.1.100) ---

# Step 1: Find your IP address
hostname -I | awk '{print $1}'
# Should show: 192.168.1.100 (your actual IP will differ)

# Step 2: Open firewall ports (Ubuntu/Debian with ufw)
sudo ufw allow 9001/tcp comment "V2V Vehicle A"
sudo ufw allow 5001/tcp comment "Vehicle A Dashboard"
sudo ufw reload
sudo ufw status

# Step 3: Generate certificates (done on Laptop A only)
cd v2v_security
python ca/ca.py
python ca/issue_cert.py --vehicle-id vehicle-a --output-dir ca/certs
python ca/issue_cert.py --vehicle-id vehicle-b --output-dir ca/certs

# Step 4: Copy Vehicle B certs to Laptop B (replace 192.168.1.101
# with B's IP)
scp ca/certs/ca_cert.pem          user@192.168.1.101:~/v2v_security/
ca/certs/
scp ca/certs/vehicle_b_cert.pem   user@192.168.1.101:~/v2v_security/
ca/certs/
scp ca/certs/vehicle_b_key.pem    user@192.168.1.101:~/v2v_security/
ca/certs/
```

```

# Step 5: Start Vehicle A (wait for connection from B)
python node/vehicle_node.py \
    --vehicle-id vehicle-a \
    --port 9001 \
    --cert ca/certs/vehicle_a_cert.pem \
    --key ca/certs/vehicle_a_key.pem \
    --ca-cert ca/certs/ca_cert.pem \
    --dashboard-port 5001

# =====
# --- On LAPTOP B (192.168.1.101) ---
# =====

# Step 1: Open firewall ports
sudo ufw allow 9002/tcp comment "V2V Vehicle B"
sudo ufw allow 5002/tcp comment "Vehicle B Dashboard"

# Step 2: Start Vehicle B (connects to Laptop A)
python node/vehicle_node.py \
    --vehicle-id vehicle-b \
    --port 9002 \
    --cert ca/certs/vehicle_b_cert.pem \
    --key ca/certs/vehicle_b_key.pem \
    --ca-cert ca/certs/ca_cert.pem \
    --peer-host 192.168.1.100 \      # Laptop A's IP address
    --peer-port 9001 \
    --dashboard-port 5002

```

Listing 22.1: Complete two-laptop setup guide

22.2 Troubleshooting Common Issues

Table 22.1: Common Errors and Solutions

Error	Cause	Fix
Connection refused	Port not open / wrong IP	Check <code>ufw status</code> ; verify IP with <code>ip addr</code>
Certificate verification failed	Wrong CA cert	Ensure same <code>ca_cert.pem</code> on both laptops
Invalid key for cert	Cert/key mismatch	Regenerate with <code>issue_cert.py</code>
Address already in use	Port taken	<code>lsof -i :9001</code> then kill that PID
Nonce replay errors	System clock skew	<code>sudo ntpdate pool.ntp.org</code> on both laptops
HMAC verification failed	Wrong session key	Check auth completed successfully before BSM exchange

Part IV

Reference and Appendices

Appendix A

Glossary

AES	Advanced Encryption Standard. A symmetric block cipher standardised by NIST (FIPS 197). The most widely used encryption algorithm in the world.
AES-GCM	AES in Galois/Counter Mode. Provides both encryption and authentication (AEAD — Authenticated Encryption with Associated Data).
Authentication	The process of verifying that a party is who it claims to be.
BSM	Basic Safety Message. Defined in SAE J2735. The standard data format for V2V safety messages. Broadcast 10 times per second.
CA	Certificate Authority. A trusted third party that issues and signs digital certificates.
CIA Triad	Confidentiality, Integrity, Availability. The three core properties of information security.
C-V2X	Cellular Vehicle-to-Everything. V2V/V2X communication using cellular (LTE/5G) technology.
DSRC	Dedicated Short-Range Communications. V2V communication standard based on IEEE 802.11p, operating at 5.9 GHz.
ECC	Elliptic Curve Cryptography. Public-key cryptography based on the algebraic structure of elliptic curves. Provides equivalent security to RSA with much smaller key sizes.
ECDH	Elliptic Curve Diffie-Hellman. A key agreement protocol that allows two parties to derive a shared secret over a public channel.
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral. ECDH with fresh key pairs per session, providing forward secrecy.
ECDSA	Elliptic Curve Digital Signature Algorithm. Used to create and verify digital signatures.
Forward Secrecy	A property ensuring that compromise of a long-term key does not compromise past session keys. Achieved via ECDHE.

HKDF	HMAC-based Key Derivation Function. Takes a shared secret and derives one or more cryptographic keys.
HMAC	Hash-based Message Authentication Code. Provides both integrity and authentication using a shared secret key.
IEEE 1609.2	The IEEE standard defining security services for wireless communications in DSRC-based V2X systems. Specifies certificate formats, signature algorithms, and trust management.
Man-in-the-Middle (MitM)	An attack where an adversary secretly relays (and possibly modifies) communications between two parties who believe they are communicating directly.
MQTT	Message Queuing Telemetry Transport. A lightweight publish/-subscribe messaging protocol designed for IoT and M2M communication.
Non-repudiation	The guarantee that a party cannot deny having performed an action (e.g., sending a message). Achieved with digital signatures.
Nonce	Number Used Once. A random value that may only appear once in a cryptographic protocol, preventing replay attacks.
PEM	Privacy Enhanced Mail. A Base64-encoded format for storing cryptographic keys and certificates.
PKI	Public Key Infrastructure. A framework of hardware, software, policies, and procedures for managing digital certificates and public-key encryption.
Pseudonymous Certificate	A digital certificate that identifies a vehicle by a temporary, unlinkable pseudonym rather than a real identifier.
Replay Attack	An attack where a valid message is captured and retransmitted at a later time to achieve a malicious effect.
RSA	Rivest–Shamir–Adleman. A public-key cryptosystem based on the difficulty of factoring large integers.
SAE J2735	SAE International standard defining the message set (including BSMs) for V2V/V2X communication.
Session Key	A short-lived symmetric key used for the duration of one communication session.
SHA-256	Secure Hash Algorithm 256-bit. A cryptographic hash function producing a 256-bit digest. Part of the SHA-2 family (FIPS 180-4).
Sybil Attack	An attack where a single adversary creates multiple fake identities to gain disproportionate influence in a distributed network.
TLS	Transport Layer Security. The cryptographic protocol providing security over the internet (used in HTTPS). Our V2V protocol

	implements similar concepts at the application layer.
V2V	Vehicle-to-Vehicle communication. Direct wireless communication between nearby vehicles without infrastructure.
VANET	Vehicular Ad Hoc Network. A network formed by vehicles and roadside infrastructure, communicating wirelessly without fixed topology.
X.509	An ITU-T standard defining the format of public-key certificates. The basis of PKI on the internet and in V2V systems.

Appendix B

Standards References

Table B.1: Key V2V Security Standards

Standard	Body	Description
IEEE 802.11p	IEEE	WAVE (Wireless Access in Vehicular Environments) — the radio layer for DSRC
IEEE 1609.2	IEEE	Security services for DSRC: certificate format, signing, verification
IEEE 1609.3	IEEE	WAVE Short Message Protocol (WSMP) networking layer
SAE J2735	SAE	V2X message set dictionary including BSM format
SAE J2945	SAE	Minimum performance requirements for V2V safety systems
ETSI EN 302 637	ETSI	European V2X: Cooperative Awareness Message (CAM) standard
ETSI TS 103 097	ETSI	European V2X security header and certificate formats
3GPP TS 33.185	3GPP	Security aspects of Mission Critical Communications (LTE-V2X)
NIST FIPS 186-4	NIST	Digital Signature Standard (DSS) — specifies ECDSA
NIST FIPS 197	NIST	Advanced Encryption Standard (AES)
NIST SP 800-56A	NIST	Recommendation for Pair-Wise Key Establishment (ECDH)
NIST SP 800-131A	NIST	Transitions: Recommendation for Transitioning Cryptographic Algorithms

Appendix C

Further Reading

1. IEEE 1609.2 specification: <https://standards.ieee.org/ieee/1609.2/7443/>
2. PyCA cryptography documentation: <https://cryptography.io/en/latest/>
3. SAE J2735 standard overview: https://www.sae.org/standards/content/j2735_202309/
4. US DOT V2V safety pilot programme: <https://www.transportation.gov/av/v2x>
5. ETSI ITS standards portal: <https://www.etsi.org/technologies/automotive-intelligent-transport>
6. “Security and Privacy in V2X Communications” survey: <https://arxiv.org/abs/2003.11082>
7. Eclipse Mosquitto MQTT broker docs: <https://mosquitto.org/documentation/>
8. Flask documentation: <https://flask.palletsprojects.com/>
9. NIST post-quantum cryptography (future-proofing): <https://csrc.nist.gov/projects/post-quantum-cryptography>
10. SCMS (Security Credential Management System) — US V2X PKI: https://www.its.dot.gov/research_areas/cybersecurity/htm/scms.htm
11. V2X Cyber Threat Analysis (ENISA report): <https://www.enisa.europa.eu/publications/enisa-threat-landscape-for-connected-and-automated-mobility-systems-and-services>
12. Cryptography Engineering (Book) by Ferguson, Schneier, Kohno: https://www.schneier.com/books/cryptography_engineering/

Appendix D

Quick-Reference Command Cheat Sheet

```
# =====
# SETUP
# =====
cd v2v_security && source venv/bin/activate
pip install -r requirements.txt

# =====
# CERTIFICATE AUTHORITY
# =====
python ca/ca.py                                # Create root CA
python ca/issue_cert.py --vehicle-id vehicle-a \
    --output-dir ca/certs                        # Issue cert to
    Vehicle A
python ca/issue_cert.py --vehicle-id vehicle-b \
    --output-dir ca/certs                        # Issue cert to
    Vehicle B
python ca/verify_cert.py \
    --cert ca/certs/vehicle_a_cert.pem \
    --ca-cert ca/certs/ca_cert.pem              # Verify a
    certificate

# =====
# RUNNING VEHICLES (same machine)
# =====
# Terminal 1 (Vehicle A)
python node/vehicle_node.py --vehicle-id vehicle-a \
    --port 9001 --cert ca/certs/vehicle_a_cert.pem \
    --key ca/certs/vehicle_a_key.pem \
    --ca-cert ca/certs/ca_cert.pem --dashboard-port 5001

# Terminal 2 (Vehicle B)
```

```

python node/vehicle_node.py --vehicle-id vehicle-b \
    --port 9002 --cert ca/certs/vehicle_b_cert.pem \
    --key ca/certs/vehicle_b_key.pem \
    --ca-cert ca/certs/ca_cert.pem \
    --peer-host 127.0.0.1 --peer-port 9001 --dashboard-port 5002

# =====
# DASHBOARDS
# =====
# http://localhost:5001    Vehicle A dashboard
# http://localhost:5002    Vehicle B dashboard

# =====
# ATTACK DEMONSTRATIONS
# =====
python attacks/replay_attack.py \
    --vehicle-a-port 9001 --vehicle-b-host 127.0.0.1 \
    --vehicle-b-port 9002 --delay 10
python attacks/tamper_attack.py \
    --listen-port 9003 --vehicle-b-host 127.0.0.1 --vehicle-b-port
    9002
python attacks/mitm_attack.py \
    --listen-port 9004 --vehicle-b-host 127.0.0.1 --vehicle-b-port
    9002

# =====
# SECURITY EVALUATION
# =====
python tests/evaluate_security.py                                # Full security
    report
pytest tests/ -v --cov=. --cov-report=html                        # Run all tests

# =====
# DOCKER
# =====
docker compose build
docker compose up -d
docker compose logs -f
docker compose down

# =====
# MQTT TESTING
# =====
mosquitto -v                                                    # Start broker
mosquitto_sub -h localhost -t "v2v/#" -v                        # Monitor all
    topics
mosquitto_pub -h localhost -t "v2v/test" -m "hello" # Test publish

```



```
# =====
# TWO-LAPTOP: FIREWALL
# =====
sudo ufw allow 9001/tcp && sudo ufw allow 9002/tcp
sudo ufw allow 5001/tcp && sudo ufw allow 5002/tcp
sudo ufw reload && sudo ufw status

# =====
# CLEANUP
# =====
deactivate                                # Deactivate
    venv
rm -rf ca/certs/                          # Remove all
    certs
```

Listing D.1: All project commands in one place

Appendix E

Troubleshooting Guide

Error: `ConnectionRefusedError: [Errno 111] Connection refused`

Cause: The peer vehicle node is not running yet, or the wrong port.

Fix: Start the server (Vehicle A) before the client (Vehicle B). Check ports match (-port on A matches -peer-port on B). Verify with: `netstat -tlnp | grep 9001`

Error: `certificate verify failed: unable to get issuer certificate`

Cause: Vehicle is using a different CA certificate than the one that signed the peer's certificate.

Fix: Ensure both vehicles use the same `ca_cert.pem`. Copy the file from the CA machine to all vehicles.

Error: `Address already in use`

Cause: Another process is using the port.

Fix: `lsof -i :9001` to find the process, then `kill $(lsof -ti:9001)` to kill it.

Error: `cryptography.exceptions.InvalidTag`

Cause: AES-GCM decryption failed — the ciphertext or tag was modified, OR the wrong session key is being used.

Fix: Verify authentication completed successfully before sending BSMs. Check that both sides derived the session key from the same ECDH exchange.

Error: `StaleMessageError: timestamp too old`

Cause: System clocks on two laptops are out of sync by more than 5 seconds.

Fix: Synchronise clocks: `sudo ntpdate pool.ntp.org`

Error: `ReplayError: nonce already seen`

Cause: Either a replay attack was detected (correct behaviour!) or the nonce generation has a bug (should not happen with `os.urandom()`).

Fix: If this appears during normal operation (not an attack test), check nonce generation code — ensure `os.urandom(32)` is called for each new message.

Error: `ModuleNotFoundError: No module named 'cryptography'`

Cause: Virtual environment not activated, or packages not installed.

Fix: `source venv/bin/activate` then `pip install -r requirements.txt`

Flask dashboard shows blank/Loading...

Cause: Flask server is not running, or CORS issue.

Fix: Ensure `-dashboard-port` is set and check the browser console (F12) for errors. Try `http://127.0.0.1:5001` instead of `localhost:5001`.

Docker: Cannot connect to the Docker daemon

Cause: Docker service not running.

Fix: `sudo systemctl start docker` (Linux) or start Docker Desktop (Windows/macOS).

Bibliography

- [1] IEEE Standards Association. *IEEE Standard for Wireless Access in Vehicular Environments Security Services for Applications and Management Messages (IEEE 1609.2-2022)*. IEEE, 2022. <https://standards.ieee.org/ieee/1609.2/7443/>
- [2] SAE International. *SAE J2735: Dedicated Short-Range Communications (DSRC) Message Set Dictionary*. SAE International, 2023. https://www.sae.org/standards/content/j2735_202309/
- [3] National Institute of Standards and Technology. *FIPS PUB 197: Advanced Encryption Standard (AES)*. NIST, 2001. <https://csrc.nist.gov/publications/detail/fips/197/final>
- [4] National Institute of Standards and Technology. *FIPS PUB 186-4: Digital Signature Standard (DSS)*. NIST, 2013. <https://csrc.nist.gov/publications/detail/fips/186/4/final>
- [5] National Institute of Standards and Technology. *NIST SP 800-56A Rev. 3: Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*. NIST, 2018. <https://csrc.nist.gov/publications/detail/sp/800-56a/rev-3/final>
- [6] Python Cryptographic Authority. *cryptography — Cryptographic Recipes and Primitives for Python*. PyCA, 2024. <https://cryptography.io/en/latest/>
- [7] V. Kumar, J. Petit, and W. Whyte. *Binary Hash Tree Based Certificate Access Management for Vehicle-to-Vehicle Communications*. In Proceedings of the 7th ACM SIGSAC Conference on Computer and Communications Security, 2015.
- [8] M. Raya and J.-P. Hubaux. *Securing Vehicular Ad Hoc Networks*. Journal of Computer Security, 15(1):39–68, 2007.
- [9] ETSI Technical Committee. *ETSI EN 302 637-2 V1.4.1: Intelligent Transport Systems (ITS); Vehicular Communications; Basic Set of Applications; Part 2: Specification of Cooperative Awareness Basic Service*. ETSI, 2019.

- [10] Eclipse Foundation. *Eclipse Mosquitto: An Open Source MQTT Broker*. Eclipse Foundation, 2024. <https://mosquitto.org/>
- [11] N. Ferguson, B. Schneier, and T. Kohno. *Cryptography Engineering: Design Principles and Practical Applications*. Wiley, 2010. ISBN: 978-0-470-47424-2.
- [12] A. Ronacher et al. *Flask: A Lightweight WSGI Web Application Framework*. Pallets Projects, 2024. <https://flask.palletsprojects.com/>
- [13] ENISA — European Union Agency for Cybersecurity. *ENISA Threat Landscape for Connected and Automated Mobility Systems and Services*. ENISA, 2023. <https://www.enisa.europa.eu/>
- [14] United States Department of Transportation. *Vehicle-to-Everything (V2X) Communications for Safety*. USDOT, 2024. <https://www.transportation.gov/av/v2x>